

리눅스 강의

1. 리눅스 강의

1-1. 리눅스란?

1-2. 리눅스의 구조

1-2-1. Atmega128에서 리눅스 구조로의 발전

1-3. 왜 리눅스인가?

2. 운영체제와 커널

2-1. OS 운영 체제란?

2-2. 커널이란?

3. 프로세스 관리

3-1. 프로세스 관리란?

3-2. 프로세서 상태 관리

프로세스 상태의 주요 구성 요소

프로세스 상태 전이

프로세스 상태 관리의 중요성

3-3. 프로세스 계층 구조

3-4. 여러 프로세스 실행 하기

터미널 다중 실행 예제

간단한 백그라운드 프로세스 실행

tmux를 사용한 다중 프로세스 실행

데몬 프로세스 실행 예제

systemd 서비스로 데몬 관리하기

리눅스 프로세스 관리 기법 비교

1. 기본 백그라운드 실행 (&)

2. nohup 사용

3. screen/tmux 사용

4. systemd 서비스

5. 프로세스 관리자 (예: Supervisor)

프로세스 관리와 환경 변수 주의사항

1. & (백그라운드) 실행 시 환경 변수

2. nohup 실행 시 환경 변수

3. systemd 서비스의 환경 변수

4. 환경 변수 관리 모범 사례

4. 커널 프로그래밍

1. 리눅스 커널 환경과 메모리 관리의 차이

2. ioremap의 역할

(1) MMU와 가상 메모리 관리

(2) 메모리 보호와 안정성

(3) 캐시 제어

(4) 플랫폼 독립성

5. 배쉬 스크립팅

5.1 Bash 스크립팅 소개.

5.1.1 Bash 란 무엇인가?

5.1.2 Bash 스크립팅의 활용 사례

5.1.3 Bash 와 다른 셸의 차이점

5.2 Bash 기본 개념

5.2.1 터미널과 명령어 기초

5.2.2 Bash 의 장점

5.2.3 스크립트 파일 생성 및 실행 방법

5.2.3 주석 작성법

5.3 변수와 데이터 처리

5.3.1 변수 선언 및 사용

5.3.2 환경변수와 사용자 정의 변수

5.3.3 문자열 조작(자르기, 치환 등)

문자열 자르기

문자열 치환

문자열 길이

실습

[5.3.4 읽기 전용 변수와 unset 명령어](#)
[읽기 전용 변수](#)
[변수 삭제: unset](#)
[실습](#)
[5.4.1 read 명령어로 입력 받기](#)
[read란?](#)
[기본 사용법](#)
[옵션 활용](#)
[여러 변수에 입력](#)
[실습](#)
[5.4.2 출력 포매팅](#)
[echo 명령어](#)
[printf 명령어](#)
[echo vs printf](#)
[실습](#)
[5.4.3 스크립트에 인자 전달하기 \(\\$1, \\$2, \\$# 등\)](#)
[스크립트 인자란?](#)
[기본 사용법](#)
[특수 변수](#)
[인자 활용 예제](#)
[실습](#)
[종합 실습](#)
[5.5 제어문](#)
[5.5.1 조건문 \(if, elif, else\)](#)
[if 문이란?](#)
[기본 사용법](#)
[else 추가](#)
[elif로 다중 조건](#)
[주의사항](#)
[실습](#)
[5.5.2 비교 연산자 \(숫자, 문자열 비교\)](#)
[숫자 비교](#)
[문자열 비교](#)
[실습](#)
[5.5.3 논리 연산 \(&&, ||\)](#)
[논리 연산자란?](#)
[사용법](#)
[실습](#)
[5.5.4 case 문 사용법](#)
[case 문이란?](#)
[기본 사용법](#)
[패턴 활용](#)
[실습](#)
[종합 실습](#)
[추가 팁](#)
[5.6 반복문](#)
[5.6.1 for 루프](#)
[for 루프란?](#)
[기본 사용법](#)
[숫자 범위 사용](#)
[실습](#)
[5.6.2. while 루프](#)
[while 루프란?](#)
[기본 사용법](#)
[입력과 결합](#)
[실습](#)
[5.6.3. until 루프](#)
[until 루프란?](#)
[기본 사용법](#)
[while과의 차이](#)
[실습](#)

실습
3. 디버깅 팁과 흔한 실수 피하기
디버깅 팁
흔한 실수와 해결법
실습
종합 실습
추가 팁
9.10 종합 실습
9.10.1. 파일 백업 스크립트 (Backup Script)
2. 시스템 모니터링 스크립트 (System Monitoring Script)
3. 사용자 입력 계산기 (Simple Calculator Script)
4. 파일 정리 스크립트 (File Cleanup Script)
5. 로그 파일 분석 스크립트 (Log File Analysis Script)
9.11 크론잡으로 스크립트 자동화
9.11.1. 크론잡이란?
2. 실제 예제: 자동 백업 스크립트와 크론잡 설정
백업 스크립트 (backup.sh)
크론잡 설정
3. 크론 사용법 상세 설명
크론탭(Crontab) 기본 구조
크론탭 예제
크론 명령어
크론 로그 확인
크론 사용 시 주의점
4. 실습 과제
5. 강의 마무리
추가 팁

1. 리눅스 강의

1-1. 리눅스란?

리눅스는 1991년 핀란드의 대학생이었던 리누스 토르발스가 개발한 운영체제입니다. 그는 미닉스(MINIX)라는 유닉스 클론 시스템에서 영감을 받아 개발을 시작했습니다.

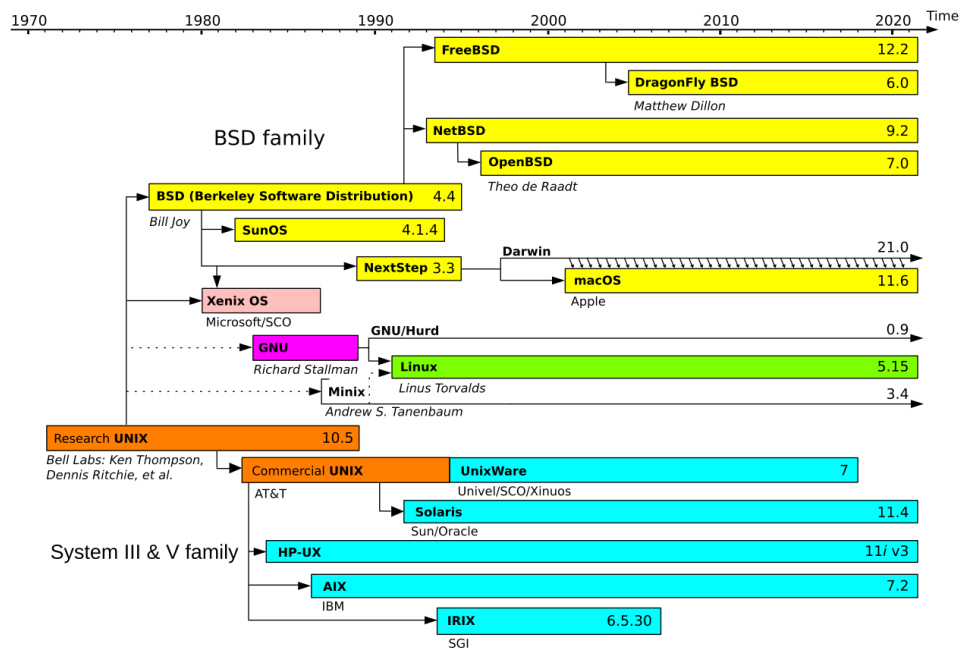
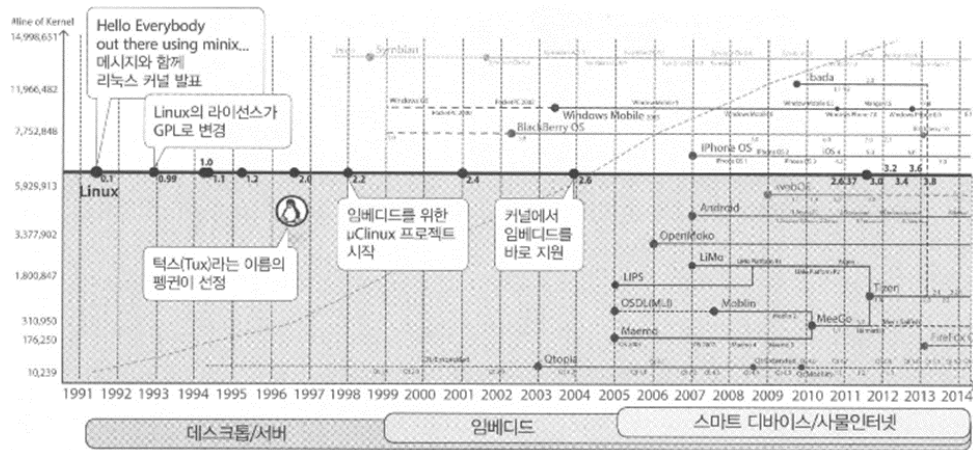
리눅스의 주요 발전 과정:

- 1991년: 리누스 토르발스가 첫 버전의 리눅스 커널 코드를 공개
- 1992년: GPL 라이선스 채택으로 자유 소프트웨어로 전환
- 1993년: 슬랙웨어(Slackware)를 시작으로 다양한 리눅스 배포판 등장
- 1994년: 버전 1.0 출시, 상용 리눅스 배포판의 시작
- 1996년: 리눅스 마스코트 'Tux' 펭귄 채택

리눅스의 특징적인 발전 요소들:

- **오픈소스 철학:** 전 세계 개발자들의 자발적 참여로 지속적인 발전
- **다양한 배포판:** Ubuntu, Red Hat, Debian 등 다양한 목적의 배포판 출현
- **서버 시장 점유:** 웹 서버, 클라우드 인프라 등에서 높은 점유율 달성
- **모바일 확장:** 안드로이드의 기반이 되어 모바일 시장에서도 영향력 확대

현재 리눅스는 서버, 임베디드 시스템, 모바일 기기 등 다양한 분야에서 사용되며, 지속적으로 발전하고 있는 운영체제입니다.



리눅스는 유닉스의 표준 규정인 POSIX(Portable Operating System Interface)를 지원 하기 때문에 다른 유닉스에서 개발된 애플리케이션을 쉽게 사용할 수 있으며, X 윈도 우같은 유닉스의 표준 GUI 시스템이 지원된다. 그리고 유닉스의 네트워크(BSD 소켓), IPC(Inter-Process Communication)나 프로세스 스케줄링(시분할 처리), 버퍼 캐시 (buffer cache)와 페이징(paging), 스레드 (POSIX thread) 등도 지원한다.

유닉스는 여러 사용자들이 동시에 사용할 수 있는데, 각각의 사용자는 자신의 계정

(user account)과 비밀번호, 그리고 홈 디렉터리(home directory)라는 저장공간을 가지 고 있다. 유닉스는 사용자 계정과 비밀번호를 이용해서 사용자를 구분하며, 계정 별로 접근 권한의 제약을 둬으로써 기본적인 시스템 보안을 제공한다.

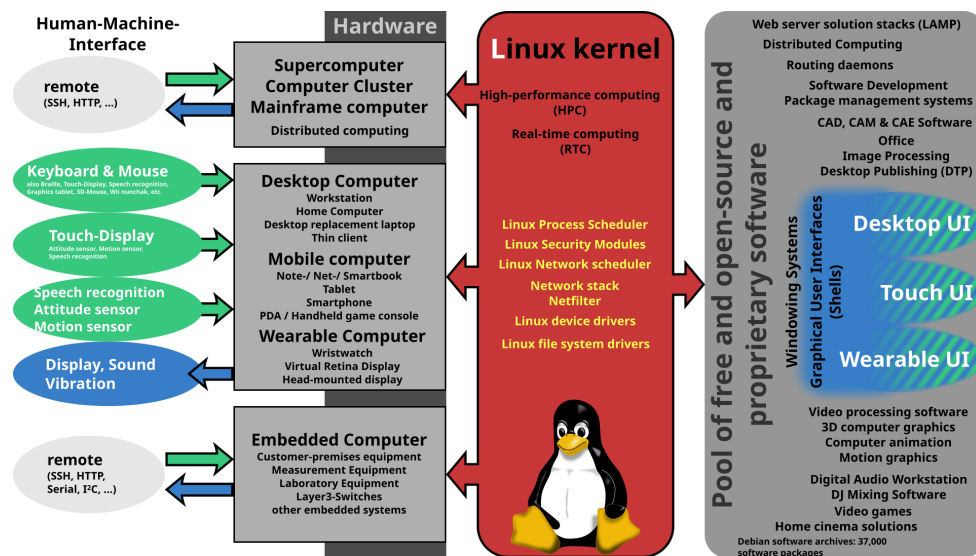
https://ko.wikipedia.org/wiki/%EB%A6%AC%EB%88%85%EC%8A%A4_%EB%B0%B0%ED%8F%AC%ED%8C%90

1-2. 리눅스의 구조

리눅스는 리누스 토발즈에 의해서 개발된 커널 자체를 의미하기도 하지만 다양한 GNU 소프트웨어가 합쳐진 운영체제를 의미하기도 한다. 운영체제는 기본적으로 컴퓨터 자원의 효율적인 사용성과 사용자가 보다 쉽게 사용할 수 있는 편의성을 제공해야 하는데, 우분투와 같은 리눅스 배포판에서 리눅스 커널과 GNU 소프트웨어를 통해서 이러한 기능을 제공한다.



리눅스의 구조를 이해하기 위해서는 각 계층이 어떻게 상호작용하는지 이해하는 것이 중요합니다. 하드웨어 계층에서부터 시작하여 커널, 시스템 라이브러리, 그리고 최상위의 응용 프로그램까지 각 계층은 서로 유기적으로 연결되어 있습니다. 이러한 계층적 구조는 리눅스가 안정적이고 효율적인 운영체제로 기능할 수 있게 하는 핵심 요소입니다.



리눅스의 구조는 크게 커널 영역과 사용자 영역으로 나눌 수 있습니다. 커널은 하드웨어 자원을 관리하고 프로세스 관리, 메모리 관리, 파일 시스템 관리 등의 핵심적인 기능을 수행합니다. 사용자 영역에서는 셸(Shell), 컴파일러, 에디터와 같은 다양한 응용 프로그램들이 실행됩니다.

1-2-1. Atmega128에서 리눅스 구조로의 발전

1. 기본 하드웨어 구조

Atmega128은 8비트 마이크로컨트롤러로, 단순한 CPU와 메모리 구조를 가지고 있습니다:

- **CPU:** 단일 코어, RISC 아키텍처
- **메모리:** Flash Memory, SRAM, EEPROM의 단순한 구조
- **주변장치:** 기본적인 I/O 포트, 타이머, ADC 등

Atmega128의 주요 한계점들:

- **제한된 처리 능력:**
 - 단일 코어로 인한 병렬 처리 불가능
 - 낮은 클럭 속도로 인한 성능 제약
 - 복잡한 연산 처리의 어려움
- **메모리 제약:**
 - 128KB Flash Memory의 제한된 용량
 - 4KB SRAM으로 인한 실행 시 메모리 부족

- 동적 메모리 할당의 어려움
- **확장성 문제:**
 - 제한된 I/O 포트 수
 - 복잡한 주변장치 연결의 어려움
 - 네트워크 기능 구현의 제약

이러한 한계점으로 인해 더 복잡하고 고성능이 요구되는 시스템에서는 리눅스와 같은 진보된 운영체제의 필요성이 대두되었습니다.

2. 리눅스로의 발전

리눅스는 이러한 기본 구조에서 다음과 같은 요소들이 추가되었습니다:

- **CPU 관리 발전:**
 - 멀티코어 프로세서 지원
 - 고급 프로세스 스케줄링
 - 실시간 처리 기능
- **메모리 관리 발전:**
 - 가상 메모리 시스템 도입
 - 페이징과 스와핑 구현
 - 동적 메모리 할당
 - 커널영역과 유저 영역(이중 모드, 시스템 호출)
- **파일 시스템:**
 - 계층적 파일 시스템 구조
 - 다양한 파일 시스템 지원 (ext4, btrfs 등)
 - 네트워크 파일 시스템

3. 현대적 특징

리눅스는 다음과 같은 현대적 기능들을 추가하여 더욱 발전했습니다:

- **보안 기능:**
 - 사용자 권한 관리
 - SELinux와 같은 보안 모듈
 - 암호화 시스템
- **네트워크 기능:**
 - TCP/IP 스택 구현
 - 네트워크 프로토콜 지원
 - 소켓 프로그래밍 인터페이스

이러한 발전을 통해 리눅스는 단순한 마이크로컨트롤러 수준에서 현대적인 운영체제로 성장했으며, 서버에서 모바일 기기까지 다양한 플랫폼에서 사용되고 있습니다.

1-3. 왜 리눅스인가?

- 과거에는 많은 임베디드 시스템들이 운영체제 없이 개발되었지만, 최근에는 대부분의 임베디드 시스템들이 어떤 형태든 운영체제를 가지고 개발되는 추세
- 요즘에는 산업용보다 개인용 장비에 더 많은 임베디드 운영체제들이 사용되고 있다. 최근 스마트폰과 태블릿 열풍으로 인기 있는 운영체제로는 Apple의 iOS, Google의 Android를 비롯하여 Meego, Limo, Tizen, BrewMP 등 다양

- 개발환경이 기본적으로 제공되기 때문에 무료로 제공하는 GNU 개발 도구는 리눅스에서 커널을 포함한 모든 응용 프로그램을 개발하는데 사용할 수 있음.
- GNU 개발 도구는 가장 많이 사용되는 PC 환경 뿐만 아니라 다른 모든 상용 Unix 시스템에서 사용이 가능
- 또한 현존하는 모든 CPU를 대상으로 실행 코드를 생성하고, 디버깅 환경을 제공함으로써 임베디드 시스템이 요구하는 크로스 개발을 지원
- 리눅스 커널은 사용자에게 필요한 기능과 하드웨어 장치에 따라 최적화된 구성으로 만들어져 메모리 사용량을 최소화 할 수 있음.
- 또한 대부분의 장치 드라이버들은 커널 실행 중에 추가 또는 제거될 수 있도록 작성되어 있어 확장이 용이

리눅스는 현대 컴퓨팅 환경에서 선택이 아닌 필수적인 운영체제로 자리잡았습니다. 그 이유는 리눅스가 가진 다재다능한 특성에 있습니다. 서버와 개인용 컴퓨터부터 우주 항공, 의료 장비와 같은 첨단 임베디드 시스템에 이르기까지, 리눅스는 광범위한 활용이 가능한 운영체제입니다.

특히 개발자들에게 리눅스는 완벽한 개발 환경을 제공합니다. GNU 개발 도구를 통해 커널은 물론 모든 종류의 응용 프로그램을 무료로 개발할 수 있으며, 이는 PC 환경뿐만 아니라 모든 상용 Unix 시스템에서도 사용이 가능합니다. 더불어 현존하는 모든 CPU를 대상으로 실행 코드를 생성하고 디버깅 환경을 제공하여, 임베디드 시스템 개발에 필수적인 크로스 개발을 완벽하게 지원합니다.

리눅스의 또 다른 강점은 최적화와 효율성입니다. 필요한 기능과 하드웨어 장치에 따라 최적화된 구성이 가능하여 메모리 사용을 최소화할 수 있으며, 시스템 운영 중에도 장치 드라이버를 자유롭게 추가하거나 제거할 수 있어 시스템 확장성이 뛰어납니다. 이러한 특성은 현대 임베디드 시스템 개발 트렌드와도 완벽하게 부합하며, 이는 리눅스가 미래 기술 발전의 핵심 기반이 될 것임을 보여줍니다.

2. 운영체제와 커널

2-1. OS 운영 체제란?

운영체제(OS)의 필요성은 컴퓨터 하드웨어와 소프트웨어의 발전 과정에서 자연스럽게 대두되었습니다. 초기 컴퓨터 시스템에서는 운영체제가 존재하지 않았으며, 프로그래머가 직접 하드웨어를 제어해야 했습니다.

1940년대 초기 컴퓨터는 진공관을 사용했으며, 프로그램 실행을 위해서는 물리적으로 배선을 다시 연결해야 했습니다. 1950년대에 들어서면서 펀치카드 시스템이 도입되었지만, 여전히 한 번에 하나의 프로그램만 실행할 수 있었고, 프로그래머가 모든 하드웨어 제어를 담당해야 했습니다.

시스템이 복잡해지고 다양한 주변장치가 등장하면서, 다음과 같은 문제들이 발생했습니다:

- **하드웨어 복잡성 증가:** 새로운 주변장치와 메모리 관리의 복잡성이 증가하면서, 프로그래머가 모든 하드웨어를 직접 제어하는 것이 불가능해졌습니다.
- **자원 관리의 비효율성:** CPU와 메모리 같은 고가의 자원이 프로그램 전환 시간 동안 낭비되는 문제가 발생했습니다.
- **프로그래밍 생산성 저하:** 각 프로그래머가 하드웨어 제어 코드를 중복해서 작성해야 하는 비효율이 발생했습니다.

이러한 문제들을 해결하기 위해, 1960년대부터 운영체제가 본격적으로 개발되기 시작했습니다. 초기의 배치 처리 시스템을 시작으로, 다중 프로그래밍, 시분할 시스템 등이 차례로 등장하면서 운영체제는 점차 발전했습니다.

현대의 운영체제는 다음과 같은 핵심적인 역할을 수행합니다:

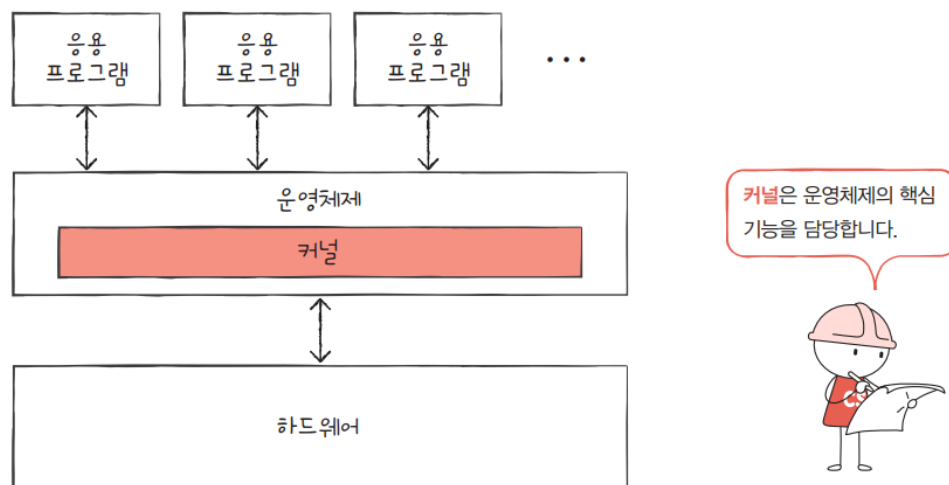
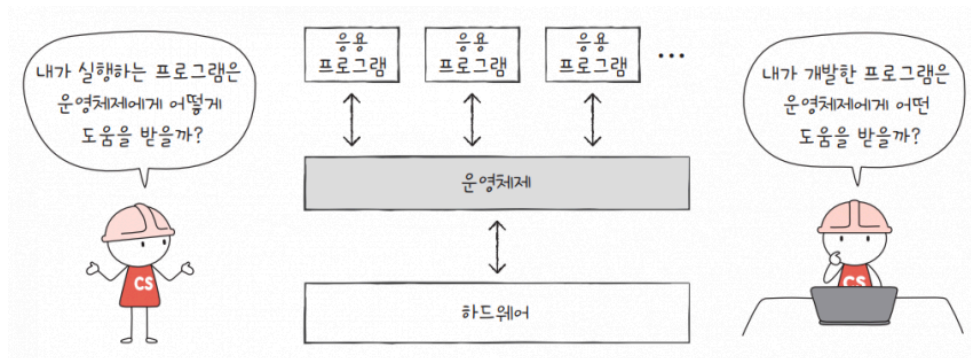
- **추상화 계층 제공:** 복잡한 하드웨어를 간단한 인터페이스로 추상화하여 프로그래머의 부담을 줄입니다.
- **자원 관리자:** 시스템의 모든 자원을 효율적으로 관리하고 할당합니다.
- **서비스 제공자:** 파일 시스템, 프로세스 관리, 네트워킹 등 기본적인 서비스를 제공합니다.

이러한 운영체제의 발전은 컴퓨터 시스템의 효율성과 사용성을 크게 향상시켰으며, 현대 컴퓨팅의 기반이 되었습니다.

결국 중요한 것은 복잡해지는 자원관리와 다중 프로세스를 처리할 수 있는 기능입니다.

2-2. 커널이란?

자원에 접근하고 조작하는 기능, 프로그램이 올바르게 안전하게 실행되게 하는 기능이 운영체제의 핵심 서비스에 속합니다. 이러한 운영체제의 핵심 서비스를 담당하는 부분을 **커널(kernel)**이라고 합니다.



운영체제는 사용자가 실행하는 응용 프로그램이 하드웨어 자원에 직접 접근하는 것을 방지하여 자원을 보호합니다. 만약 응용 프로그램이 CPU, 메모리, 하드 디스크 등에 마음대로 접근하고 조작할 수 있다면 자원이 무질서하게 관리될 것이고, 응용 프로그램이 조금만 실수해도 컴퓨터 전체에 큰 악영향을 끼칠 수 있습니다.

그래서 운영체제는 응용 프로그램들이 자원에 접근하려고 할 때 오직 자신을 통해서만 접근하도록 하여 자원을 보호합니다. 비유하자면 운영체제는 응용 프로그램의 자원 접근을 대행하는 일종의 문지기 역할을 하는 셈입니다. 응용 프로그램이 자원에 접근하기 위해서는 운영체제에 도움을 요청해야 합니다. 이때 '운영체제에 도움을 요청한다'는 말은 '운영체제 코드를 실행하려고 한다'는 말과 같습니다. 응용 프로그램의 요청을 받은 운영체제는 응용 프로그램 대신 자원에 접근하여 요청한 작업을 수행합니다. 거기에 권한 레벨을 통해 보호되고 있는 메모리 영역에 접근하는 것을 의미합니다.

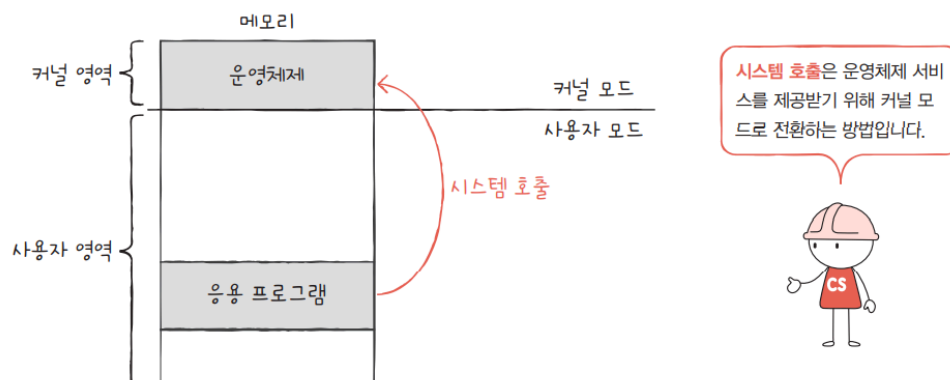


이러한 운영체제의 문지기 역할은 이중 모드로서 구현됩니다. **이중 모드**dual mode란 CPU가 명령어를 실행하는 모드를 크게 사용자 모드와 커널 모드로 구분하는 방식입니다. CPU는 명령어를 사용자 모드로써 실행할 수 있고, 커널 모드로써 실행할 수 있습니다.

사용자 모드user mode는 운영체제 서비스를 제공받을 수 없는 실행 모드입니다. 즉, 커널 영역의 코드를 실행할 수 없는 모드입니다. 일반적인 응용 프로그램은 기본적으로 사용자 모드로 실행됩니다. 사용자 모드로 실행 중인 CPU는 입출력 명령어와 같이 하드웨어 자원에 접근하는 명령어를 실행할 수 없습니다. 그래서 사용자 모드로 실행되는 일반적인 응용 프로그램은 자원에 접근할 수 없습니다.

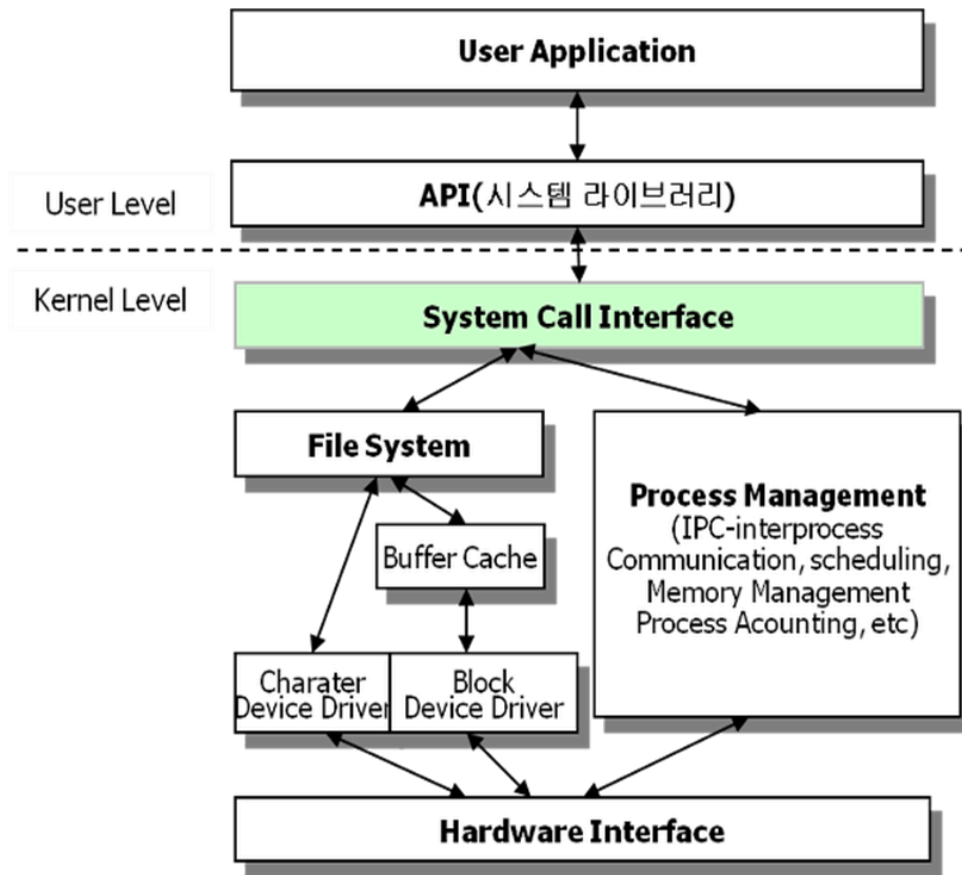
반면 **커널 모드**kernel mode는 운영체제 서비스를 제공받을 수 있는 실행 모드입니다. 즉, 커널 영역의 코드를 실행할 수 있는 모드입니다. CPU가 커널 모드로 명령어를 실행하면 자원에 접근하는 명령어를 비롯한 모든 명령어를 실행할 수 있습니다. 운영체제는 커널 모드로 실행되기 때문에 자원에 접근할 수 있습니다.

응용 프로그램이 커널 모드로 전환되어야 하는 경우, 시스템 호출(System Call)이라는 특별한 인터페이스를 통해 운영체제에 서비스를 요청합니다. 이러한 모드 전환은 하드웨어적으로 보호되어 있어, 악의적인 프로그램이 임의로 커널 모드에 접근하는 것을 방지합니다. 따라서 이중 모드는 시스템의 안정성과 보안성을 보장하는 핵심 메커니즘이라고 할 수 있습니다.

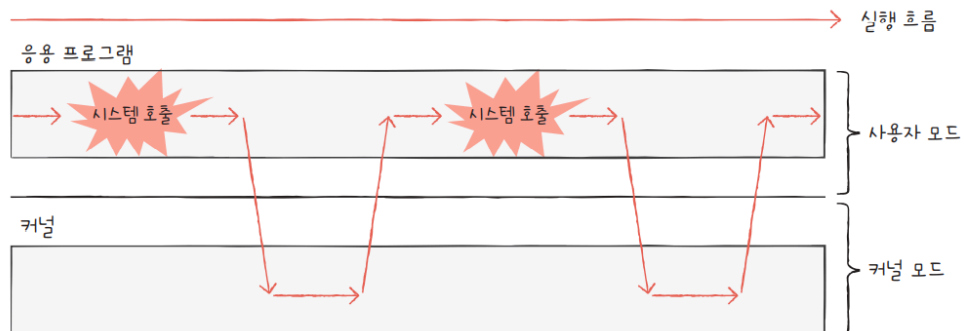


시스템 호출은 일종의 인터럽트입니다. 정확히는 소프트웨어적인 인터럽트입니다. 인터럽트는 입출력장치에 의해 발생하기도 하지만 인터럽트를 발생시키는 특정 명령어에 의해 발생하기도 하는데, 이를 **소프트웨어 인터럽트**라고 합니다.

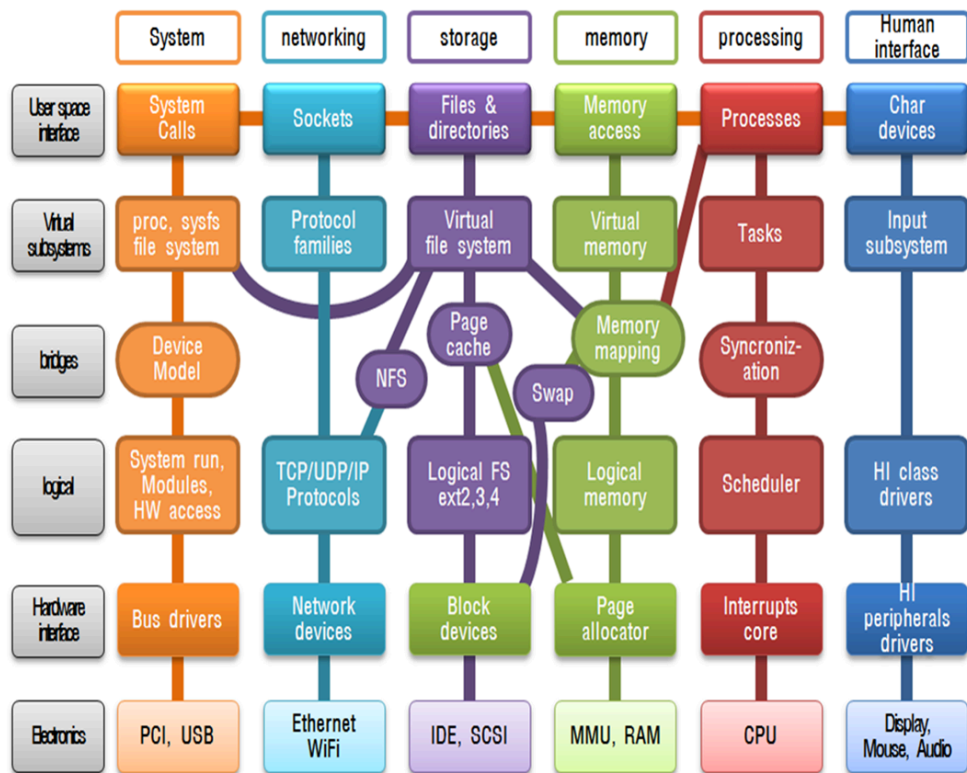
그래서 CPU가 시스템 호출을 처리하는 순서는 인터럽트 처리 순서와 유사합니다. 시스템 호출을 발생시키는 명령어가 실행되면 CPU는 지금까지의 작업을 백업하고, 커널 영역 내에 시스템 호출을 수행하는 코드(인터럽트 서비스 루틴)를 실행한 뒤 다시 기존에 실행하던 응용 프로그램으로 복귀하여 실행을 계속해 나갑니다.



일반적으로 응용 프로그램은 실행 과정에서 운영체제 서비스들을 매우 빈번하게 이용합니다. 그 과정에서 빈번하게 시스템 호출을 발생 시키고 사용자 모드와 커널 모드를 오가며 실행됩니다.



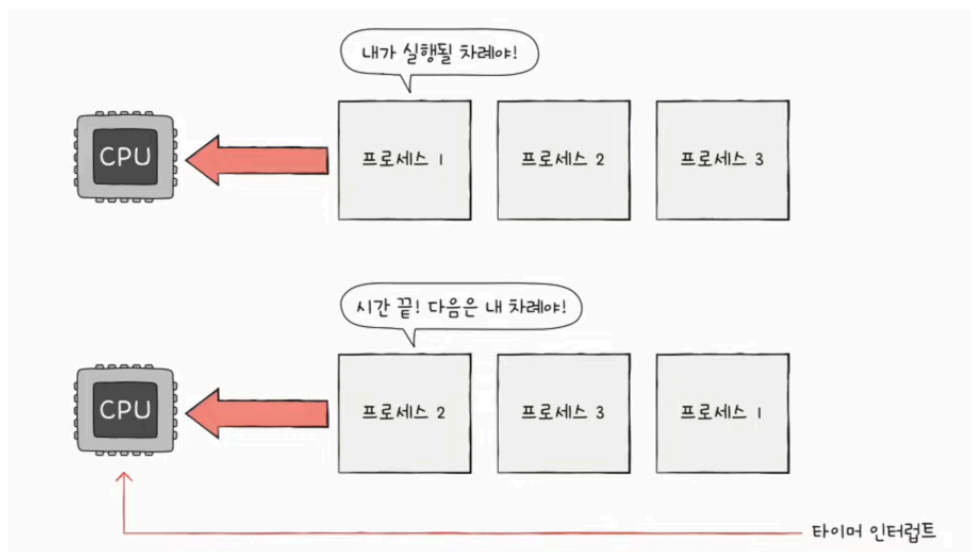
[example] strace 를 이용해서 ls 를 관찰하기



3. 프로세스 관리

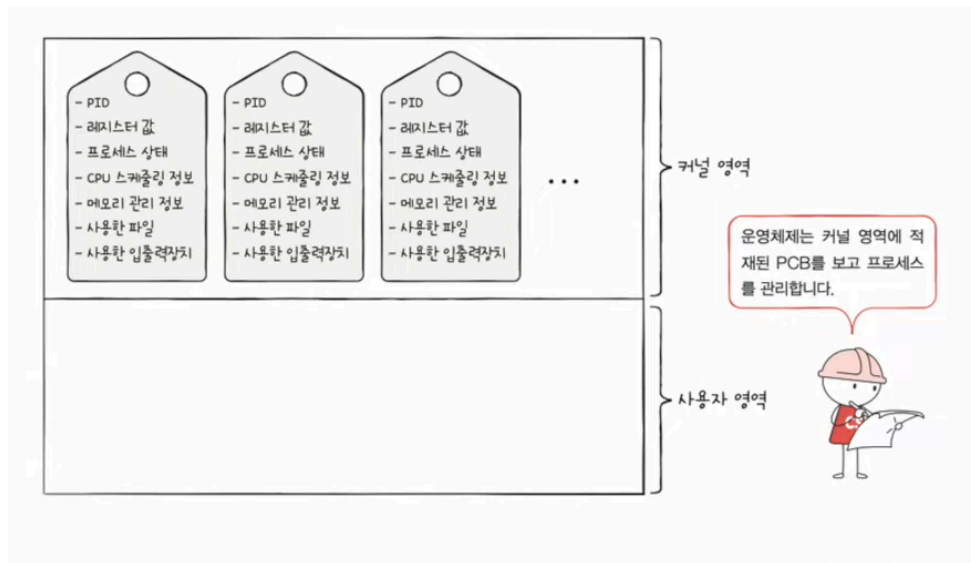
3-1. 프로세스 관리란?

리눅스 커널은 CFS(Completely Fair Scheduler)를 사용하여 프로세스에 CPU 시간을 공정하게 분배합니다. 각 프로세스는 우선순위와 실행 시간을 기반으로 스케줄링됩니다.



리눅스에서 PCB(Process control block)는 **프로세스 제어 블록**으로, 각 프로세스의 상태와 정보를 저장하는 커널 데이터 구조입니다. 리눅스에서는 PCB가 task_struct 구조체로 구현됩니다. task_struct는 리눅스 커널 소스 코드의 include/linux/sched.h에 정의

되어 있으며, 프로세스와 스레드(리눅스에서는 경량 프로세스로 취급)를 모두 관리합니다.



```
// include/linux/sched.h (리눅스 커널 5.15 기준)

struct task_struct {
    // 프로세스 상태
    volatile long state; // TASK_RUNNING, TASK_INTERRUPTIBLE 등
    void *stack;        // 프로세스 스택 포인터

    // 프로세스 식별 정보
    pid_t pid;          // 프로세스 ID
    pid_t tgid;         // 스레드 그룹 ID (프로세스 그룹 리더의 PID)
    struct task_struct *group_leader; // 스레드 그룹 리더

    // 부모-자식 관계
    struct task_struct *real_parent; // 실제 부모 프로세스
    struct task_struct *parent;      // 현재 부모 프로세스 (보통 real_parent와 동일)
    struct list_head children;       // 자식 프로세스 리스트
    struct list_head sibling;        // 형제 프로세스 리스트

    // 스케줄링 정보
    int prio;          // 정적 우선순위
    int static_prio;   // 기본 우선순위
    int normal_prio;   // 동적 우선순위
    unsigned int rt_priority; // 실시간 우선순위
    const struct sched_class *sched_class; // 스케줄링 클래스 (CFS, RT 등)
    struct sched_entity se; // CFS 스케줄링 엔터티
    struct sched_rt_entity rt; // 실시간 스케줄링 엔터티
    unsigned long rt_timeout; // 실시간 타임아웃

    // 메모리 관리
    struct mm_struct *mm; // 메모리 디스크립터 (가상 메모리 관리)
    struct mm_struct *active_mm; // 현재 활성화된 메모리 디스크립터

    // 파일 디스크립터
    struct files_struct *files; // 열린 파일 디스크립터 테이블

    // 시그널 처리
```

```

struct signal_struct *signal; // 시그널 관련 정보
struct sighand_struct *sighand; // 시그널 핸들러
sigset_t blocked; // 차단된 시그널 집합
sigset_t real_blocked; // 실제 차단된 시그널 집합
struct sigpending pending; // 대기 중인 시그널

// CPU 상태
struct thread_struct thread; // CPU별 스레드 정보 (레지스터 상태 등)
unsigned long *cpu_context; // CPU 컨텍스트

// 리소스 사용량
struct rusage ru; // 리소스 사용 통계
cputime_t utime, stime; // 사용자 시간, 시스템 시간
unsigned long nvcsw, nivcsw; // 자발적/비자발적 컨텍스트 스위치 횟수

// 기타
char comm[TASK_COMM_LEN]; // 실행 파일 이름 (예: "bash")
struct fs_struct *fs; // 파일 시스템 정보 (루트 디렉토리, 현재 디렉토리)
struct cred *cred; // 프로세스 권한 정보 (UID, GID 등)
};

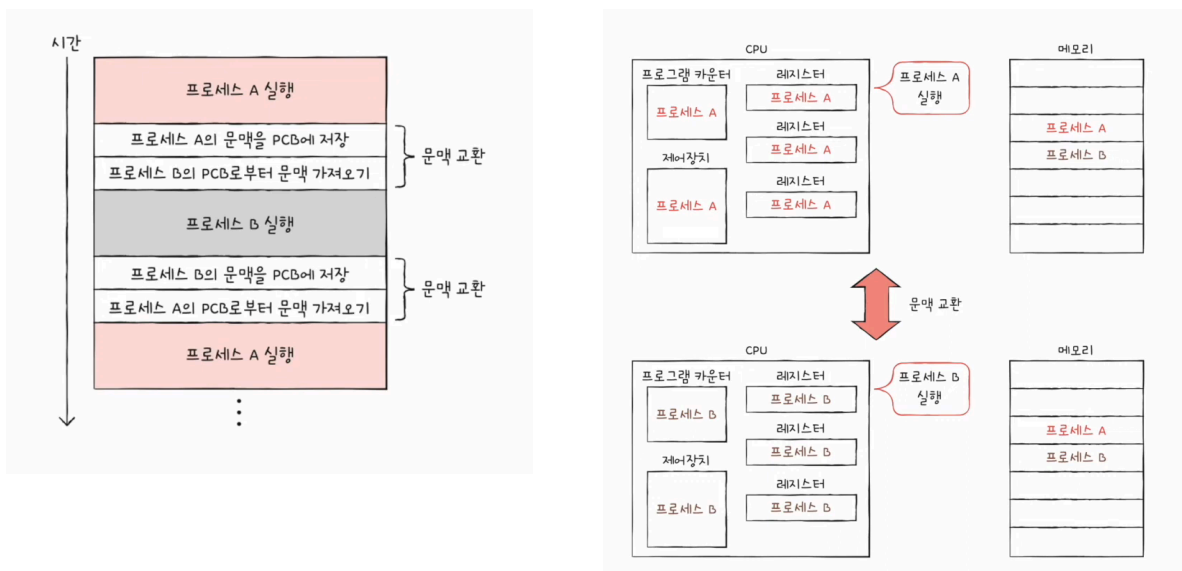
```

프로세스 문맥 교환(Context Switch)은 CPU가 한 프로세스에서 다른 프로세스로 전환될 때 발생하는 중요한 작업입니다. 이 때 현재 실행 중인 프로세스의 상태를 저장하고, 다음 실행할 프로세스의 상태를 복원해야 합니다.

문맥 교환 시 저장/복원되는 주요 정보:

- **CPU 레지스터 값:** 프로그램 카운터(PC), 스택 포인터(SP) 등의 레지스터 상태
- **메모리 관리 정보:** 페이지 테이블, 메모리 매핑 등
- **프로세스 상태 정보:** PCB(Process Control Block)에 저장된 프로세스 관련 정보
- **I/O 상태 정보:** 열린 파일, 네트워크 연결 등의 I/O 관련 상태

문맥 교환은 상당한 시스템 오버헤드를 발생시키므로, 운영체제는 이를 최소화하기 위해 효율적인 스케줄링 알고리즘을 사용합니다. 특히 리눅스의 CFS(Completely Fair Scheduler)는 문맥 교환의 빈도와 비용을 고려하여 프로세스 스케줄링을 수행합니다.



3-2. 프로세서 상태 관리

프로세서의 상태 관리는 운영체제의 핵심 기능 중 하나로, 시스템의 안정성과 효율성을 보장하는 중요한 역할을 합니다.

프로세스 상태의 주요 구성 요소

- **TASK_RUNNING**: 프로세스가 CPU에서 실행 중이거나 실행 대기 중인 상태
- **TASK_INTERRUPTIBLE**: 프로세스가 특정 조건이나 이벤트를 기다리는 수면 상태 (시그널로 깨울 수 있음)
- **TASK_UNINTERRUPTIBLE**: 인터럽트 불가능한 수면 상태 (주로 I/O 작업 중)
- **TASK_ZOMBIE**: 종료된 프로세스지만 부모가 wait() 호출하기 전 상태
- **TASK_STOPPED**: SIGSTOP 등의 시그널로 일시 중지된 상태

프로세스 상태 전이

프로세스는 다음과 같은 상황에서 상태가 변경됩니다:

- **schedule()**: TASK_RUNNING → CPU 실행 (CFS 스케줄러에 의해 선택)
- **preempt_schedule()**: CPU 실행 → TASK_RUNNING (선점 스케줄링)
- **sys_wait()**: TASK_ZOMBIE → 완전 종료 (부모 프로세스가 자원 회수)
- **wake_up_process()**: TASK_INTERRUPTIBLE → TASK_RUNNING (프로세스 깨우기)

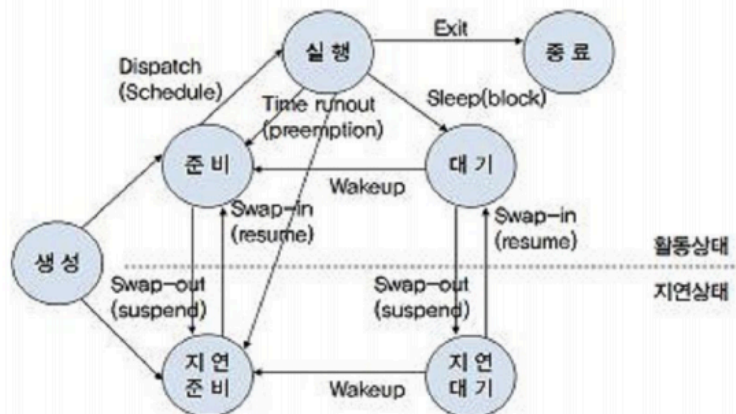
프로세스 상태 관리의 중요성

효율적인 프로세스 상태 관리는 다음과 같은 이점을 제공합니다:

- **시스템 성능 최적화**: nice 값과 실시간 우선순위를 통한 CPU 스케줄링 최적화
- **시스템 자원 관리**: OOM(Out Of Memory) killer를 통한 메모리 자원 보호
- **프로세스 동기화**: futex와 세마포어를 통한 프로세스 간 동기화
- **시스템 안정성**: watchdog을 통한 시스템 행들 감지 및 복구

커널은 이러한 상태 관리를 통해 시스템의 안정성을 유지하고, 사용자 프로세스들이 효율적으로 실행될 수 있는 환경을 제공합니다.

프로세스 상태 전이도



3-3. 프로세스 계층 구조

[실습]pstree 명령어

1. 부모 프로세스와 자식 프로세스란?

- **부모 프로세스(Parent Process):**

- 새로운 프로세스를 생성하는 프로세스를 부모 프로세스라고 합니다.
- 부모 프로세스는 자식 프로세스를 생성하고 자식의 종료 상태를 수거합니다.
- 예: 셸(bash)이 사용자 명령어를 실행할 때 부모 프로세스가 되어 자식 프로세스를 생성합니다.

- **자식 프로세스(Child Process):**

- 부모 프로세스가 생성한 새로운 프로세스를 자식 프로세스라고 합니다.
- 자식 프로세스는 부모로부터 자원(메모리, 파일 디스크립터 등)을 상속받습니다.
- 자식 프로세스는 독립적으로 실행되며, 부모와 구별되는 고유한 PID(프로세스 ID)를 가집니다.

2. 부모-자식 관계의 특징

- **PID와 PPID:**

- 각 프로세스는 고유한 PID를 가지며, 부모의 PID는 자식 프로세스에 PPID(Parent PID)로 저장됩니다.
- 예: `ps -ef` 명령어로 PPID 컬럼을 통해 부모 프로세스를 확인할 수 있습니다.

- **자원 상속:**

- 자식 프로세스는 부모의 메모리, 파일 디스크립터, 환경 변수 등을 상속받습니다.
- 단, 메모리는 **Copy-on-Write(COW)** 방식으로 공유되며, 수정 시에만 독립된 복사본이 생성됩니다.

- **종료와 상태 수거:**

- 자식 프로세스가 종료되면 `EXIT_ZOMBIE` 상태가 되어 부모가 `wait()` 또는 `waitpid()`로 종료 상태를 수거할 때까지 대기합니다.
- 부모가 종료되면 자식 프로세스는 ****고아 프로세스(Orphan Process)****가 되어 `init` 프로세스(PID 1) 또는 `systemd`가 관리합니다.

- **프로세스 트리:**

- 리눅스의 모든 프로세스는 트리 구조로 관리됩니다. `init`(또는 `systemd`, PID 1)이 최상위 프로세스이며, 모든 프로세스는 여기서 파생됩니다.
- 예: `ps tree` 명령어로 전체 프로세스 트리를 확인할 수 있습니다.

4. fork-exec 생성 기법

리눅스에서 프로세스를 생성하고 새로운 프로그램을 실행하는 가장 일반적인 방법은 `fork()`와 `exec()` 시스템 콜을 조합하는 **fork-exec** 기법입니다. 이 기법은 부모 프로세스가 자식 프로세스를 생성한 후, 자식이 새로운 프로그램으로 대체되는 방식으로 동작합니다.

5. fork() 시스템 콜

- **역할:**

- `fork()`는 현재 프로세스를 복제하여 새로운 자식 프로세스를 생성합니다.
- 부모와 자식은 거의 동일한 상태(메모리, 파일 디스크립터 등)를 가지지만, 별개의 프로세스로 독립적으로 실행됩니다.

- **반환값:**

- 부모 프로세스: 자식 프로세스의 PID를 반환.
- 자식 프로세스: 0을 반환.
- 오류 시: -1을 반환.

- **특징:**

- 자식 프로세스는 부모의 메모리를 **Copy-on-Write(COW)** 방식으로 공유합니다. 즉, 메모리 페이지가 수정되기 전까지는 동일한 물리 메모리를 참조하며, 수정 시 복사본이 생성됩니다.
- 자식은 부모의 파일 디스크립터, 환경 변수, 시그널 핸들러 등을 상속받습니다.

6. exec() 시스템 콜

- **역할:**
 - `exec()` 계열 함수는 현재 프로세스의 실행 이미지를 새로운 프로그램으로 교체합니다.
 - 메모리, 코드, 데이터, 스택 등이 새로운 프로그램으로 대체되지만, PID와 파일 디스크립터는 유지됩니다.
- **주요 함수:**
 - `execl()`, `execle()`, `execv()`, `execvp()` 등.
 - 예: `execl("/bin/ls", "ls", NULL)`는 `ls` 프로그램을 실행.
- **특징:**
 - `exec()`은 성공 시 반환하지 않으며, 실패 시 -1을 반환.
 - 열린 파일 디스크립터는 기본적으로 유지되지만, `close-on-exec` 플래그가 설정된 경우 닫힘.
 - 시그널 핸들러와 환경 변수는 초기화될 수 있음.

7. fork-exec 기법의 전체 흐름

`fork()`와 `exec()`를 조합하여 새로운 프로그램을 실행하는 과정은 다음과 같습니다:

- **부모 프로세스가 `fork()` 호출:**
 - 자식 프로세스를 생성.
 - 자식은 부모의 메모리, 파일 디스크립터 등을 상속받음.
- **자식 프로세스가 `exec()` 호출:**
 - 자식 프로세스의 실행 이미지를 새로운 프로그램으로 교체.
 - PID와 파일 디스크립터는 유지됨.
- **부모 프로세스가 `wait()` 호출:**
 - 자식 프로세스가 종료될 때까지 대기.
 - 자식의 종료 상태를 수거.

8. fork-exec 기법의 장점과 단점

- **장점**
 - **유연성:**
 - `fork()` 후 자식 프로세스에서 환경 설정(파일 디스크립터, 환경 변수 등)을 변경한 후 `exec()`를 호출할 수 있음.
 - 예: 파일 디스크립터 리다이렉션(`dup2()`) 후 `exec()` 호출.
 - **안정성:**
 - 부모와 자식이 독립적으로 실행되므로, 부모 프로세스는 자식의 실행에 영향을 받지 않음.
 - **표준화:**
 - 유닉스/리눅스에서 표준적으로 사용되는 프로세스 생성 방식.
- **단점**
 - **오버헤드:**
 - `fork()`는 부모 프로세스를 복제하므로 메모리 복사(COW 포함)에 오버헤드가 발생.
 - `exec()`가 호출되면 복제된 메모리가 폐기되고 새로운 이미지가 로드되므로 비효율적일 수 있음.
 - **복잡성:**
 - `fork()`와 `exec()`를 별도로 호출해야 하므로 코드가 복잡해질 수 있음.
- **대안: `posix_spawn()`**

`fork-exec`의 오버헤드를 줄이기 위해 `posix_spawn()` 함수가 제공됩니다. 이 함수는 `fork()`와 `exec()`를 단일 시스템 콜로 결합하여 효율적으로 프로세스를 생성합니다.

3-4. 여러 프로세스 실행 하기

리눅스에서 여러 프로세스를 동시에 실행하고 관리하는 실습을 해보겠습니다.

터미널 다중 실행 예제

```
#!/bin/bash

# 새로운 터미널에서 명령어를 실행하는 함수
open_terminal() {
    gnome-terminal -- bash -c "$1; exec bash"
}

# 첫 번째 터미널: top 명령어 실행
open_terminal "top"

# 두 번째 터미널: htop 실행 (설치되어 있는 경우)
open_terminal "htop"

# 세 번째 터미널: 디스크 사용량을 1초마다 모니터링
open_terminal "while true; do df -h; sleep 1; clear; done"

# 네 번째 터미널: 시스템 로그 모니터링
open_terminal "tail -f /var/log/syslog"
```

이 스크립트를 실행하면 다음과 같은 작업이 수행됩니다:

- 각각 다른 모니터링 도구를 실행하는 4개의 터미널 창이 열립니다.
- 첫 번째 창에서는 시스템 자원 사용량을 보여주는 top 명령어가 실행됩니다.
- 두 번째 창에서는 더 향상된 시스템 모니터링 도구인 htop이 실행됩니다.
- 세 번째 창에서는 디스크 사용량이 1초마다 업데이트되어 표시됩니다.
- 네 번째 창에서는 시스템 로그를 실시간으로 모니터링합니다.

스크립트 사용 방법:

1. 위 코드를 terminal_multi.sh 파일로 저장합니다.
2. 실행 권한을 부여합니다: `chmod +x terminal_multi.sh`
3. 스크립트를 실행합니다: `./terminal_multi.sh`

참고: 이 스크립트는 GNOME 터미널을 사용하는 환경에서 동작합니다. 다른 터미널 에뮬레이터를 사용하는 경우 gnome-terminal 대신 해당하는 명령어로 변경해야 합니다.

간단한 백그라운드 프로세스 실습

```
python script.py &

nohup python script.py > output.log 2>&1 &
```

다음은 기본적인 백그라운드 프로세스 실행 예제입니다:

```
#!/bin/bash
```

```
# 1. 단순 백그라운드 프로세스 실행
echo "현재 시간을 매 초마다 기록" > time.log
while true; do
    date >> time.log
    sleep 1
done &
echo "백그라운드 프로세스 PID: $!"

# 2. 여러 백그라운드 프로세스 실행
for i in {1..3}; do
    (echo "프로세스 $i 시작"; sleep 10; echo "프로세스 $i, 종료") &
done

# 3. 백그라운드 프로세스 목록 확인
jobs

# 4. 프로세스 종료하기
kill %1 # 첫 번째 백그라운드 프로세스 종료
```

이 스크립트는 다음과 같은 작업을 수행합니다:

- 시간을 매 초마다 기록하는 무한 루프를 백그라운드로 실행
- 3개의 프로세스를 동시에 백그라운드로 실행하고 10초 후 종료
- 실행 중인 백그라운드 프로세스 목록 확인
- 특정 백그라운드 프로세스 종료

실행 방법:

1. 스크립트를 background_test.sh로 저장
2. 실행 권한 부여: `chmod +x background_test.sh`
3. 스크립트 실행: `./background_test.sh`

백그라운드 프로세스는 & 기호를 사용하여 실행하며, jobs 명령어로 관리하고 kill 명령어로 종료할 수 있습니다.

tmux를 사용한 다중 프로세스 실행

tmux는 터미널 멀티플렉서로, 여러 터미널 세션을 효율적으로 관리할 수 있게 해주는 도구입니다. 다음은 tmux를 사용하여 여러 프로세스를 동시에 실행하고 모니터링하는 예제입니다:

```
#!/bin/bash

# tmux 세션 생성
tmux new-session -d -s monitor_session

# 첫 번째 창에서 top 실행
tmux send-keys -t monitor_session:0 'top' C-m

# 새로운 창 생성하고 htop 실행
tmux new-window -t monitor_session:1
tmux send-keys -t monitor_session:1 'htop' C-m

# 시스템 로그 모니터링을 위한 창 생성
tmux new-window -t monitor_session:2
tmux send-keys -t monitor_session:2 'tail -f /var/log/syslog' C-m
```

```
# 창을 수직으로 분할하고 디스크 사용량 모니터링
tmux split-window -v -t monitor_session:2
tmux send-keys -t monitor_session:2.1 'watch df -h' C-m

# tmux 세션에 연결
tmux attach-session -t monitor_session
```

이 스크립트는 다음과 같은 작업을 수행합니다:

- **세션 생성:** 'monitor_session'이라는 이름의 새로운 tmux 세션을 생성합니다.
- **다중 창 관리:** 여러 개의 창을 생성하고 각각 다른 모니터링 도구를 실행합니다.
- **창 분할:** 마지막 창을 수직으로 분할하여 두 개의 모니터링 도구를 동시에 표시합니다.

tmux 기본 조작 방법:

- **Ctrl+b c** : 새 창 생성
- **Ctrl+b n** : 다음 창으로 이동
- **Ctrl+b p** : 이전 창으로 이동
- **Ctrl+b %** : 창을 수직으로 분할
- **Ctrl+b "** : 창을 수평으로 분할
- **Ctrl+b d** : tmux 세션 분리(detach)

세션을 분리한 후에도 프로세스들은 백그라운드에서 계속 실행됩니다. 나중에 `tmux attach -t monitor_session` 명령어로 다시 연결할 수 있습니다.

데몬 프로세스 실행 예제

데몬(Daemon)은 백그라운드에서 실행되는 프로세스입니다. 다음은 간단한 로그 기록 데몬을 만드는 예제입니다:

```
#!/bin/bash

# 데몬 프로세스로 실행할 로그 기록 프로그램
log_daemon() {
    # 작업 디렉토리를 루트로 변경
    cd /

    # 표준 입출력과 에러를 /dev/null로 리다이렉션
    exec 1>/dev/null
    exec 2>/dev/null
    exec 0</dev/null

    # 새로운 세션 생성
    setsid

    # 매 5초마다 시스템 정보를 로그 파일에 기록
    while true; do
        date >> /tmp/system_monitor.log
        uptime >> /tmp/system_monitor.log
        echo "-----" >> /tmp/system_monitor.log
        sleep 5
    done
}

# 데몬 프로세스를 백그라운드로 실행
```

```
log_daemon &
```

```
# 데몬의 PID 출력
```

```
echo "데몬 프로세스가 시작되었습니다. PID: $!"
```

이 스크립트는 다음과 같은 특징을 가집니다:

- **백그라운드 실행:** 프로세스가 터미널과 독립적으로 실행됩니다.
- **표준 스트림 리다이렉션:** 모든 입출력을 `/dev/null`로 리다이렉션하여 터미널과 분리합니다.
- **새로운 세션:** `setsid`를 통해 새로운 세션을 생성합니다.
- **주기적인 작업:** 5초마다 시스템 정보를 로그 파일에 기록합니다.

스크립트 사용 방법:

1. 위 코드를 `daemon_example.sh` 파일로 저장합니다.
2. 실행 권한을 부여합니다: `chmod +x daemon_example.sh`
3. 스크립트를 실행합니다: `./daemon_example.sh`
4. 로그 확인: `tail -f /tmp/system_monitor.log`

참고: 실제 프로덕션 환경에서는 `systemd` 서비스를 사용하여 데몬을 관리하는 것이 더 권장됩니다.

systemd 서비스로 데몬 관리하기

`systemd`를 사용하여 데몬 프로세스를 관리하는 방법을 알아보겠습니다.

1. 먼저 데몬 스크립트를 작성합니다 (`/usr/local/bin/mydaemon.sh`):

```
#!/bin/bash

# 로그 디렉토리 생성
mkdir -p /var/log/mydaemon

while true; do
    # 시스템 정보를 로그에 기록
    date >> /var/log/mydaemon/system_info.log
    uptime >> /var/log/mydaemon/system_info.log
    echo "-----" >> /var/log/mydaemon/system_info.log
    sleep 5
done
```

2. 스크립트에 실행 권한을 부여합니다:

```
sudo chmod +x /usr/local/bin/mydaemon.sh
```

3. `systemd` 서비스 유닛 파일을 생성합니다 (`/etc/systemd/system/mydaemon.service`):

```
[Unit]
Description=My System Monitor Daemon
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/mydaemon.sh
Restart=always
```

```
User=root
StandardOutput=append:/var/log/mydaemon/service.log
StandardError=append:/var/log/mydaemon/error.log

[Install]
WantedBy=multi-user.target
```

4. systemd 서비스 관리 명령어:

- 서비스 활성화: `sudo systemctl enable mydaemon.service`
- 서비스 시작: `sudo systemctl start mydaemon.service`
- 서비스 상태 확인: `sudo systemctl status mydaemon.service`
- 서비스 중지: `sudo systemctl stop mydaemon.service`
- 서비스 재시작: `sudo systemctl restart mydaemon.service`

5. 로그 확인:

```
# 서비스 로그 확인
sudo journalctl -u mydaemon.service

# 데몬 로그 확인
tail -f /var/log/mydaemon/system_info.log
```

장점:

- systemd가 자동으로 프로세스 관리
- 시스템 부팅 시 자동 시작
- 로그 관리 기능 제공
- 프로세스 모니터링 및 재시작

리눅스 프로세스 관리 기법 비교

리눅스에서 여러 프로세스를 관리하는 주요 방법들을 비교해보겠습니다:

1. 기본 백그라운드 실행 (&)

- 장점:
 - 간단하고 즉시 사용 가능
 - 추가 도구 설치 불필요
 - 기본적인 작업에 적합
- 단점:
 - 프로세스 관리 기능 제한적
 - 세션 종료시 프로세스도 종료
 - 로그 관리가 어려움

2. nohup 사용

- 장점:
 - 세션 종료 후에도 프로세스 유지
 - 로그 파일 자동 생성
 - 표준 출력/에러 리다이렉션 지원

- **단점:**
 - 프로세스 모니터링이 제한적
 - 자동 재시작 기능 없음
 - 복잡한 프로세스 관리에는 부적합

3. screen/tmux 사용

- **장점:**
 - 다중 세션 관리 가능
 - 세션 분리/재접속 가능
 - 화면 분할 및 작업 관리 용이
- **단점:**
 - 추가 설치 필요
 - 학습 곡선이 있음
 - 자동화된 프로세스 관리에는 제한적

4. systemd 서비스

- **장점:**
 - 체계적인 프로세스 관리
 - 자동 시작/재시작 기능
 - 종합적인 로그 관리
 - 의존성 관리 지원
- **단점:**
 - 설정이 상대적으로 복잡
 - root 권한 필요
 - 시스템 수준의 설정 필요

5. 프로세스 관리자 (예: Supervisor)

- **장점:**
 - 웹 인터페이스 제공
 - 상세한 프로세스 모니터링
 - 그룹 단위 프로세스 관리
- **단점:**
 - 추가 설치 및 설정 필요
 - 리소스 사용량 증가
 - 관리 오버헤드 발생

기법	설치 필요	세션 유지	자동 재시작	모니터링	복잡도
&	X	X	X	X	낮음
nohup	X	O	X	X	낮음
screen/tmux	O	O	X	O	중간
systemd	X	O	O	O	높음
Supervisor	O	O	O	O	중간

권장 사용 사례:

- 간단한 백그라운드 작업: & 또는 nohup
- 개발/디버깅 환경: screen/tmux
- 프로덕션 서버: systemd 또는 Supervisor
- 시스템 서비스: systemd

프로세스 관리와 환경 변수 주의사항

프로세스를 관리할 때 환경 변수와 관련하여 주의해야 할 사항들을 알아보겠습니다.

1. & (백그라운드) 실행 시 환경 변수

- **현재 세션 종속성:** 백그라운드로 실행된 프로세스는 실행 시점의 환경 변수를 상속받습니다.
- **세션 종료 영향:** 터미널 세션이 종료되면 환경 변수와 함께 프로세스도 종료됩니다.
- **변수 변경 미반영:** 실행 후 변경된 환경 변수는 이미 실행 중인 프로세스에 영향을 주지 않습니다.

2. nohup 실행 시 환경 변수

- **실행 시점 스냅샷:** nohup으로 실행된 프로세스는 실행 시점의 환경 변수만 유지합니다.
- **export 필수:** 환경 변수는 반드시 export로 선언되어야 nohup 프로세스에 전달됩니다.
- **세션 독립성:** 터미널이 종료되어도 프로세스는 실행 시점의 환경을 유지한 채 계속 실행됩니다.

3. systemd 서비스의 환경 변수

- **.bashrc 미적용:** systemd 서비스는 기본적으로 사용자의 .bashrc에 정의된 환경 변수를 상속받지 않습니다.
- **명시적 정의 필요:** 환경 변수는 서비스 유닛 파일에서 Environment 또는 EnvironmentFile로 정의해야 합니다.
- **시스템 환경:** /etc/environment에 정의된 시스템 전역 환경 변수는 systemd 서비스에서도 사용 가능합니다.

4. 환경 변수 관리 모범 사례

- **설정 파일 사용:** 환경 변수를 별도의 설정 파일로 관리하여 유지보수성을 높입니다.
- **문서화:** 프로세스가 의존하는 환경 변수를 명확히 문서화합니다.
- **기본값 설정:** 환경 변수가 없는 경우를 대비해 프로그램 내에서 기본값을 설정합니다.

보안 주의사항:

- 민감한 정보(비밀번호, API 키 등)는 환경 변수로 전달할 때 주의가 필요합니다.
- 로그 파일에 환경 변수 값이 기록되지 않도록 주의해야 합니다.
- 프로덕션 환경에서는 환경 변수 관리를 위한 보안 도구 사용을 고려하세요.

4. 커널 프로그래밍

1. 리눅스 커널 환경과 메모리 관리의 차이

Atmega 같은 마이크로컨트롤러 환경과 리눅스 커널 환경은 메모리 관리 방식에서 근본적인 차이가 있습니다.

- **Atmega (베어 메탈 환경):**
 - 메모리 관리 유닛(MMU)이 없거나 사용되지 않는 경우가 많습니다.
 - 프로그램은 물리적 메모리 주소에 직접 접근하며, 주소 공간이 단일하고 평평합니다(flat address space).

- 따라서 `volatile uint8_t*`로 물리적 주소를 캐스팅하여 직접 읽고 쓰는 방식이 가능합니다. 예를 들어, 레지스트리 주소가 0x20이라면 `(volatile uint8_t*)0x20`으로 접근하면 됩니다.
- 이 환경에서는 메모리 보호나 가상화가 없기 때문에 단순하고 직관적입니다.
- **리눅스 커널 환경:**
 - 리눅스는 MMU를 사용하는 현대적인 운영체제입니다. 모든 메모리 접근은 가상 주소(Virtual Address)를 통해 이루어지며, 물리적 주소(Physical Address)는 직접 접근할 수 없습니다.
 - 사용자 공간과 커널 공간이 분리되어 있고, 각 프로세스는 고유한 가상 주소 공간을 가집니다.
 - 커널 모듈도 커널 공간에서 동작하지만, 여전히 가상 주소 체계를 따릅니다. 따라서 물리적 주소를 직접 사용하려면 이를 커널의 가상 주소로 매핑해야 합니다.

2. ioremap의 역할

`ioremap`은 물리적 메모리 주소(예: 하드웨어 레지스트리 주소)를 커널의 가상 주소로 매핑하는 함수입니다. 이를 사용하는 이유는 다음과 같습니다:

(1) MMU와 가상 메모리 관리

- 리눅스에서 CPU는 가상 주소만 이해합니다. MMU가 가상 주소를 물리적 주소로 변환하여 하드웨어에 접근합니다.
- 물리적 주소를 직접 사용하려고 하면, MMU가 이를 가상 주소로 해석하려다가 잘못된 메모리 영역에 접근하거나 페이지 폴트(page fault)를 일으킬 수 있습니다.
- `ioremap`은 물리적 주소를 커널 가상 주소 공간에 매핑하고, 페이지 테이블을 설정하여 MMU가 올바르게 동작하도록 보장합니다.

(2) 메모리 보호와 안정성

- 리눅스는 메모리 보호를 위해 엄격한 규칙을 적용합니다. 물리적 주소를 직접 캐스팅하여 접근하면, 커널이 이를 허용하지 않거나 예기치 않은 동작(시스템 크래시 등)이 발생할 수 있습니다.
- `ioremap`은 매핑된 영역에 대한 접근 권한을 명시적으로 설정하며, 커널이 이를 관리할 수 있게 합니다.

(3) 캐시 제어

- 하드웨어 레지스트리는 일반 메모리와 달리 캐싱되면 안 됩니다. 예를 들어, 레지스트리 값을 읽거나 쓰는 작업이 캐시에 저장되면 실제 하드웨어 상태와 불일치가 발생할 수 있습니다.
- `ioremap`은 기본적으로 "non-cacheable" 속성을 설정하여 캐시를 우회합니다(예: `ioremap_nocache`). 이는 `volatile`만으로는 보장할 수 없는 중요한 특성입니다.

(4) 플랫폼 독립성

- 서로 다른 아키텍처(ARM, x86 등)에서 메모리 매핑 방식이 다를 수 있습니다. `ioremap`은 이러한 차이를 추상화하여 플랫폼 독립적인 코드를 작성할 수 있게 합니다.
- 반면, 직접 캐스팅은 특정 하드웨어에 종속적이며 이식성이 떨어집니다.

`ioremap`은 리눅스 커널의 메모리 관리와 하드웨어 접근을 안전하고 효율적으로 처리하기 위한 필수 도구입니다. `Atmega`처럼 직접 캐스팅하는 방식은 MMU가 없는 환경에서는 유효하지만, 리눅스에서는 가상 주소 체계와 메모리 보호 때문에 불가능합니다. 커널 가상 주소로 매핑함으로써 시스템 안정성을 유지하고, 하드웨어와의 상호작용을 올바르게 보장할 수 있습니다.

`ioremap()`: 개발자가 물리적 주소와 레지스트리를 직접 다루며, GPIO를 포함한 모든 하드웨어를 세부적으로 제어할 수 있습니다. 하지만 코드가 복잡하고 플랫폼 의존적입니다.
`gpio_request()`: GPIO 서브시스템이 메모리 매핑과 레지스트리 조작을 추상화하여, 개발자는 단순히 GPIO 번호로 제어할 수 있습니다. 내부적으로 `ioremap()`과 유사한 메커니즘을 사용하지만, 이는 커널 드라이버 수준에서 처리됩니다.

- **`ioremap()`:** 개발자가 물리적 주소와 레지스트리를 직접 다루며, GPIO를 포함한 모든 하드웨어를 세부적으로 제어할 수 있습니다. 하지만 코드가 복잡하고 플랫폼 의존적입니다.
- **`gpio_request()`:** GPIO 서브시스템이 메모리 매핑과 레지스트리 조작을 추상화하여, 개발자는 단순히 GPIO 번호로 제어할 수 있습니다. 내부적으로 `ioremap()`과 유사한 메커니즘을 사용하지만, 이는 커널 드라이버 수준에서 처리됩니다.

5. 배쉬 스크립팅

5.1 Bash 스크립팅 소개.

5.1.1 Bash 란 무엇인가?

Bash(Bourne Again Shell)는 유닉스와 리눅스 시스템에서 가장 널리 사용되는 명령어 인터프리터이자 스크립팅 언어입니다. 1989년 Brian Fox가 GNU 프로젝트의 일환으로 개발했으며, Bourne Shell(sh)의 향상된 버전으로 설계되었습니다. 현재 대부분의 리눅스 배포판에서 기본 셸로 채택되어 있으며, 강력한 스크립팅 기능과 사용자 친화적인 인터페이스를 제공합니다.

5.1.2 Bash 스크립팅의 활용 사례

- 시스템 자동화
 - 서버 백업 스크립트
 - 로그 정리 및 모니터링
- 개발 환경 세팅
 - 패키지 설치 자동화 (`apt` , `yum` , `brew`)
 - 프로젝트 초기화 스크립트
- 데이터 처리
 - 파일명 일괄 변경
 - 텍스트 필터링 (grep, sed, awk 등과 연동)
- CI/CD 자동화
 - 빌드 & 배포 파이프라인에서 사용
- 교육 및 실습 자동화
 - 실습 환경 초기화 스크립트

5.1.3 Bash 와 다른 셸의 차이점

종류	설명	특징
sh	Bourne Shell	기본적인 기능, POSIX 표준
bash	Bourne Again Shell	sh 호환 + 확장 기능, 가장 널리 사용됨
zsh	Z Shell	자동 완성 강화, 테마 설정 가능
fish	Friendly Interactive Shell	사용 편의성 중심, 직관적인 문법
dash	Debian Almquist Shell	빠르고 가벼움, Ubuntu의 기본 /bin/sh

- 호환성: `sh` 스크립트는 대부분 `bash` 에서 실행 가능
- 성능: 일부 작업에서 `dash` 가 더 빠름 (init script 등)
- 사용성: `zsh` , `fish` 는 개발자 친화적 환경 제공

5.2 Bash 기본 개념

5.2.1 터미널과 명령어 기초

터미널은 사용자가 컴퓨터와 상호작용하는 텍스트 기반 인터페이스입니다. 명령어를 입력하면 시스템은 이를 해석하고 실행하여 결과를 출력합니다. 기본적인 명령어들은 다음과 같습니다:

5.2.2 Bash 의 장점

Bash 스크립팅은 Unix/Linux 시스템에서 시스템 관리, 자동화, 리소스 관리 및 일상적인 작업을 수행하는 데 활용되는 강력하고 다재 다능한 도구입니다. 셸 스크립팅의 주요 장점은 다음과 같습니다.

- **자동화:** 셸 스크립트로 반복 작업과 프로세스를 자동화하여 시간을 절약하고 수동 실행으로 인한 오류 위험을 줄일 수 있습니다.
- **이식성:** 셸 스크립트는 에뮬레이터나 가상 머신을 통해 Unix, Linux, macOS, Windows 등 다양한 운영 체제에서 실행할 수 있습니다.
- **유연성:** 셸 스크립트는 사용자의 요구사항에 맞게 쉽게 수정할 수 있으며, 다른 프로그래밍 언어나 유틸리티와 결합하여 더 강력한 기능을 구현할 수 있습니다.
- **접근성:** 셸 스크립트는 특별한 도구 없이 일반 텍스트 편집기로 작성할 수 있으며, 대부분의 운영 체제에 셸 인터프리터가 내장되어 있습니다.
- **통합:** 셸 스크립트는 데이터베이스, 웹 서버, 클라우드 서비스 등 다양한 도구와 통합되어 복잡한 자동화 및 시스템 관리 작업을 수행할 수 있습니다.
- **디버깅:** 셸 스크립트는 디버깅이 용이하며, 대부분의 셸에는 문제를 신속하게 파악하고 해결할 수 있는 디버깅 도구가 내장되어 있습니다.

5.2.3 스크립트 파일 생성 및 실행 방법

- **스크립트 명명 규칙**

Bash 스크립트 파일은 일반적으로 `.sh` 확장자로 끝납니다. 단, 확장자가 없어도 스크립트는 정상적으로 실행될 수 있습니다.

- **Shebang 추가**

Bash 스크립트는 Shebang으로 시작합니다. Shebang은 `#!/` 와 bash 셸의 경로로 구성되며, 스크립트의 첫 번째 줄에 위치합니다. 이는 시스템에게 해당 스크립트를 bash 셸로 실행하도록 지시하는 bash 인터프리터의 절대 경로입니다.

다음은 Shebang의 예시입니다.

```
#!/bin/bash
```

bash 셸의 경로는 다음 명령어로 확인할 수 있습니다.

```
which bash
```

- **첫 번째 bash 스크립트 만들기**

첫 번째 스크립트는 사용자에게 경로를 입력받아 해당 경로의 내용을 보여줍니다.

`vi` 명령어를 사용하여 `run_all.sh` 파일을 생성합니다. 원하는 다른 텍스트 편집기를 사용해도 됩니다.

```
vi run_all.sh
```

파일에 다음 내용을 입력하고 저장하세요.

```
#!/bin/bash
echo "Today is " `date`

echo -e "\nenter the path to directory"
read the_path

echo -e "\n you path has the following files and folders: "
ls $the_path
```

이제 스크립트의 각 줄을 자세히 살펴보겠습니다. 같은 내용을 줄 번호와 함께 다시 보여드리겠습니다.

```
1 #!/bin/bash
2 echo "Today is " `date`3
4 echo -e "\nenter the path to directory"
5 read the_path
6
7 echo -e "\n you path has the following files and folders: "
8 ls $the_path
```

- 1번째 줄: shebang(`#!/bin/bash`)은 bash 셸의 경로를 지정합니다.
- 2번째 줄: `echo` 명령어와 `date` 명령어로 현재 날짜와 시간을 출력합니다.
- 4번째 줄: 사용자에게 유효한 경로 입력을 요청합니다.
- 5번째 줄: `read` 명령어로 입력값을 `the_path` 변수에 저장합니다.
- 8번째 줄: `ls` 명령어로 지정된 경로의 파일과 폴더 목록을 표시합니다.

• bash 스크립트 실행

스크립트를 실행하려면 먼저 다음 명령어로 실행 권한을 부여해야 합니다.

```
chmod u+x run_all.sh
```

각 옵션의 의미:

- `chmod u` 는 현재 사용자의 파일 권한을 수정합니다.
- `+x` 는 실행 권한을 추가합니다. 이제 파일 소유자가 스크립트를 실행할 수 있습니다.
- `run_all.sh` 는 권한을 변경할 대상 파일입니다.

스크립트는 다음 방법 중 하나로 실행할 수 있습니다.

- `sh run_all.sh`
- `bash run_all.sh`
- `./run_all.sh`
- `source run_all.sh`

5.2.3 주석 작성법

Bash 스크립트에서 주석은 코드의 가독성과 유지보수성을 높이기 위해 매우 중요합니다. 한 줄 주석은 '#' 기호를 사용하여 작성하며, 여러 줄 주석은 별도의 구문이 없어 각 줄마다 '#'를 사용해야 합니다.

```
# 한 줄 주석 예시
echo "Hello World" # 이 줄은 Hello World를 출력합니다

# 여러 줄 주석 예시
# 이 스크립트는 사용자 입력을 받아
# 지정된 디렉토리의 내용을 표시하는
# 간단한 예제입니다
```

5.3 변수와 데이터 처리

5.3.1 변수 선언 및 사용

Bash 스크립트에서 변수는 데이터를 저장하고 조작하는 데 사용되는 기본적인 요소입니다. 변수는 숫자, 문자열, 배열 등 다양한 유형의 데이터를 저장할 수 있으며, 선언할 때 특별한 키워드가 필요하지 않습니다. 변수 이름은 문자, 숫자, 밑줄(_)을 포함할 수 있지만, 숫자로 시작할 수 없습니다.

변수를 선언할 때는 등호(=) 기호를 사용하며, 등호 앞뒤에 공백을 두면 안 됩니다. 변수의 값을 참조할 때는 변수 이름 앞에 달러(\$) 기호를 붙여야 합니다. 다음은 기본적인 변수 선언과 사용의 예시입니다.

```
name="John"
age=25
echo "My name is $name and I am $age years old"
```

5.3.2 환경변수와 사용자 정의 변수

환경변수는 시스템 전체에서 사용되는 전역 변수로, 시스템의 동작과 프로그램의 실행에 영향을 미치는 중요한 설정값들을 저장합니다. 반면 사용자 정의 변수는 스크립트 내에서 로컬하게 사용되는 변수로, 스크립트가 실행되는 동안에만 유효합니다. 환경변수는 대문자로 작성하는 것이 관례이며, export 명령어를 통해 설정할 수 있습니다.

주요 환경변수의 예시로는 PATH(실행 파일 경로), HOME(사용자 홈 디렉토리), USER(현재 사용자 이름) 등이 있습니다. 다음은 환경 변수의 설정과 확인 방법을 보여주는 예시입니다.

```
export MY_VARIABLE="Hello World" # 환경변수 설정
echo $MY_VARIABLE                # 환경변수 값 출력
env                               # 모든 환경변수 목록 표시
```

5.3.3 문자열 조작(자르기, 치환 등)

문자열 자르기

- \${변수:시작:길이}로 부분 문자열 추출

```
text="Hello, World"
echo ${text:0:5}    # 출력: Hello
echo ${text:7}      # 출력: World (7번째부터 끝까지)
```

문자열 치환

- \${변수/찾을문자/바꿀문자}로 치환

```
text="I like cats"
echo ${text/cats/dogs} # 출력: I like dogs
```

•

- 모든 문자열 치환: // 사용

```
text="cat cat cat"
echo ${text//cat/dog} # 출력: dog dog dog
```

문자열 길이

- \${#변수}로 길이 확인

```
text="Hello"
echo ${#text} # 출력: 5
```

실습

1. 변수 sentence="I love programming"에서 "love"를 출력해보세요.
2. "programming"을 "coding"으로 바꾸어 출력해보세요.

5.3.4 읽기 전용 변수와 unset 명령어

읽기 전용 변수

- readonly로 변수 수정 방지:

```
readonly MY_CONST="상수값"
echo $MY_CONST # 출력: 상수값
MY_CONST="변경" # 오류 발생
```

- 선언과 동시에 설정 가능:

```
readonly PI=3.14
```

변수 삭제: unset

- 변수 제거:

```
var="임시값"
unset var
echo $var # 출력: 아무것도 없음
```

- 주의: readonly 변수는 unset으로 삭제 불가.

실습

1. readonly로 COLOR="blue"를 설정하고 수정 시도해보세요.
2. 일반 변수 temp="data"를 만들고 unset으로 삭제한 뒤 출력해보세요.

5.4.1 read 명령어로 입력 받기

read란?

- 사용자로부터 입력을 받아 변수에 저장하는 명령어
- 기본 형식: read 변수명

기본 사용법

- 입력 받기:

```
echo "이름을 입력하세요: "
read name
echo "입력한 이름: $name"
```

- 실행하면 사용자의 입력을 기다립니다.

옵션 활용

- p (프롬프트 메시지 표시):

```
read -p "나이를 입력하세요: " age
echo "당신의 나이: $age"
```

- s (비밀 입력용):

```
read -s -p "비밀번호를 입력하세요: " password
echo -e "\n입력된 비밀번호: $password"
```

- r (백슬래시 이스케이프 처리 무시):

```
read -r raw_input
echo "입력: $raw_input"
```

여러 변수에 입력

- 공백으로 구분된 입력을 여러 변수에 저장:

```
read -p "이름과 나이를 입력하세요: " name age
echo "이름: $name, 나이: $age"
```

실습

1. read로 사용자의 이름을 입력받아 출력해보세요.
2. s 옵션을 사용해 비밀번호를 입력받고 확인해보세요.
3. 한 줄에 이름과 도시를 입력받아 각각 변수에 저장해보세요.

5.4.2 출력 포매팅

echo 명령어

- 간단한 출력 도구.
- 기본 사용:

```
echo "안녕하세요!"
```

- 개행 없이 출력 (-n):

```
echo -n "이어지는 " ; echo "문장"
# 출력: 이어지는 문장
```

- 이스케이프 문자 활성화 (-e):

```
echo -e "줄바꿈\n탭\t사용"
# 출력: 줄바꿈 탭 사용
```

printf 명령어

- 더 정교한 포매팅 제공.
- 형식: printf "형식문자열" 값1 값2 ...
- 예시:

```
printf "이름: %s, 나이: %d\n" "홍길동" 25
# 출력: 이름: 홍길동, 나이: 25
```

- 주요 형식 지정자:
 - %s: 문자열
 - %d: 정수
 - %f: 부동 소수점
 - \n: 줄바꿈
- 정밀도 조정:

```
printf "소수: %.2f\n" 3.14159
# 출력: 소수: 3.14
```

echo vs printf

- echo: 간단하고 빠름, 기본 개행 포함.
- printf: 세밀한 제어 가능, 개행은 수동 추가.

실습

1. echo -e로 "안녕\n세상"을 출력해보세요.
2. printf로 이름과 나이를 형식에 맞춰 출력해보세요 (예: "이름: [name], 나이: [age]").
3. printf로 소수점 두 자리까지 출력해보세요.

5.4.3 스크립트에 인자 전달하기 (\$1, \$2, \$# 등)

스크립트 인자란?

- 스크립트 실행 시 전달되는 값.
- \$1, \$2, ...로 첫 번째, 두 번째 인자를 참조.

기본 사용법

- 스크립트 예제 (test.sh):

```
#!/bin/bash
echo "첫 번째 인자: $1"
echo "두 번째 인자: $2"
```

- 실행:

```
bash test.sh "안녕" "세상"
# 출력:# 첫 번째 인자: 안녕# 두 번째 인자: 세상
```

특수 변수

- \$#: 인자 개수:

```
echo "인자 수: $#"
```

- \$0: 스크립트 이름:


```
echo "스크립트 이름: $0"
```

- \$@ 또는 \$*: 모든 인자:

```
echo "모든 인자: $@"
```

인자 활용 예제

- 스크립트 (greet.sh):

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "인자를 입력하세요!"
else
    echo "안녕, $1님!"
fi
```

- 실행:

```
bash greet.sh "홍길동"
# 출력: 안녕, 홍길동님!
```

실습

1. 스크립트를 만들어 첫 번째 인자를 출력해보세요.
2. 인자 개수(\$#)와 모든 인자(\$@)를 출력하는 스크립트를 작성해보세요.
3. 인자가 없으면 "인자 필요"를 출력하도록 수정해보세요.

종합 실습

1. 사용자로부터 이름과 나이를 read로 입력받아 변수에 저장하세요.
2. printf로 "이름: [name], 나이: [age]세" 형식으로 출력하세요.
3. 스크립트에 두 개의 인자를 전달받아, 첫 번째는 이름, 두 번째는 도시로 출력하는 스크립트를 작성하세요.
 - 예: bash script.sh "홍길동" "서울"

5.5 제어문

5.5.1 조건문 (if, elif, else)

if 문이란?

- 조건에 따라 코드 실행을 제어.
- 기본 형식:

```
if [ 조건 ]; then
    # 실행할 코드
fi
```

기본 사용법

- 예제:

```
age=20
if [ $age -gt 18 ]; then
    echo "성인입니다."
fi
```

else 추가

- 조건이 참이 아닐 때 실행:

```
age=16
if [ $age -gt 18 ]; then
    echo "성인입니다."
else
    echo "미성년자입니다."
fi
```

elif로 다중 조건

- 여러 조건을 체크:

```
age=25
if [ $age -lt 13 ]; then
    echo "어린이입니다."
elif [ $age -lt 20 ]; then
    echo "청소년입니다."
else
    echo "성인입니다."
fi
```

주의사항

- [조건]에서 공백 필수 (예: [\$age -gt 18]).
- 조건 끝에 세미콜론(;) 필요.

실습

- 변수 num에 숫자를 저장하고, 10보다 크면 "큰 수"를 출력해보세요.
- else를 추가해 10 이하일 때 "작은 수"를 출력해보세요.
- elif로 0보다 작은 경우 "음수"를 출력하도록 수정해보세요.

5.5.2 비교 연산자 (숫자, 문자열 비교)

숫자 비교

- 주요 연산자:
 - eq: 같다 (equal)
 - ne: 같지 않다 (not equal)
 - gt: 크다 (greater than)
 - lt: 작다 (less than)
 - ge: 크거나 같다 (greater or equal)
 - le: 작거나 같다 (less or equal)
- 예제:

```
num=15
if [ $num -gt 10 ]; then
    echo "10보다 큼니다."
fi
```

문자열 비교

- 주요 연산자:
 - =: 같다
 - !=: 같지 않다
 - z: 문자열 길이가 0
 - n: 문자열 길이가 0이 아님
- 예제:

```
name="홍길동"
if [ "$name" = "홍길동" ]; then
    echo "이름이 일치합니다."
fi
```

- 빈 문자열 체크:

```
input=""
if [ -z "$input" ]; then
    echo "입력값이 없습니다."
fi
```

- 주의: 변수는 따옴표로 감싸는 것이 안전 ("\$name").

실습

1. 두 숫자를 비교해 같으면 "동일", 다르면 "다름"을 출력해보세요.
2. 문자열 변수가 "yes"면 "확인", 아니면 "취소"를 출력해보세요.
3. 변수가 비어 있는지 확인하는 코드를 작성해보세요.

5.5.3 논리 연산 (&&, ||)

논리 연산자란?

- 여러 조건을 조합.
- 주요 연산자:
 - &&: AND (그리고)
 - ||: OR (또는)
 - !: NOT (부정)

사용법

- && (모두 참일 때 실행):

```
age=25
if [ $age -gt 18 ] && [ $age -lt 30 ]; then
    echo "20대입니다."
fi
```

- || (하나라도 참이면 실행):

```
day="토요일"
if [ "$day" = "토요일" ] || [ "$day" = "일요일" ]; then
    echo "주말입니다."
fi
```

- ! (조건 반전):

```
if [ ! -z "$input" ]; then
    echo "입력값이 있습니다."
fi
```

실습

1. 나이가 10보다 크고 20보다 작은 경우 "10대"를 출력해보세요.
2. 요일이 "월요일"이거나 "금요일"이면 "특별한 날"을 출력해보세요.
3. 변수가 비어 있지 않으면 "데이터 존재"를 출력해보세요.

5.5.4 case 문 사용법

case 문이란?

- 다중 조건을 간결하게 처리.
- 형식:

```
case 변수 in
    패턴1) 명령어 ;;
    패턴2) 명령어 ;;
    *) 기본 명령어 ;;
esac
```

기본 사용법

- 예제:

```
fruit="사과"
case $fruit in
    "사과") echo "빨간 과일" ;;
    "바나나") echo "노란 과일" ;;
    *) echo "알 수 없는 과일" ;;
esac
```

- : 기본값 (일치하는 패턴 없을 때).

패턴 활용

- 여러 값 처리:

```
day="월요일"
case $day in
    "월요일"|"수요일"|"금요일") echo "홀수 요일" ;;
    "화요일"|"목요일") echo "짝수 요일" ;;
    *) echo "주말" ;;
esac
```

실습

1. 변수 color에 색상을 넣고, "red"면 "빨강", "blue"면 "파랑", 그 외는 "기타"를 출력해보세요.
2. 숫자(17)를 입력받아 15는 "평일", 6~7은 "주말"을 출력하는 case 문을 작성해보세요.

종합 실습

1. 사용자로부터 숫자를 입력받아, 0보다 작으면 "음수", 0이면 "영", 0보다 크면 "양수"를 출력하세요.
2. 이름과 나이를 입력받아, 나이가 18 이상이고 이름이 "admin"이면 "관리자", 아니면 "일반 사용자"를 출력하세요.
3. 요일을 입력받아 case 문으로 "월금"은 "근무일", "토일"은 "휴일"을 출력하세요.

추가 팁

- 조건에서 변수가 비어 있을 경우 오류 방지를 위해 기본값 설정:

```
${변수:-기본값}
```

- 복잡한 조건은 (())로 산술 연산 가능:

```
if (( age > 18 && age < 30 )); then
    echo "20대"
fi
```

- case 문은 패턴 매칭에 유용 (와일드카드 *, ? 사용 가능).

5.6 반복문

5.6.1 for 루프

for 루프란?

- 일정 범위나 리스트를 순회하며 반복 실행.
- 기본 형식:

```
for 변수 in 리스트; do
    # 실행할 코드
done
```

기본 사용법

- 리스트 순회:

```
for name in "홍길동" "김영희" "이철수"; do
    echo "이름: $name"
done
```

- 출력:

```
이름: 홍길동
이름: 김영희
이름: 이철수
```

숫자 범위 사용

- {시작..끝} 활용:

```
for i in {1..5}; do
    echo "숫자: $i"
done
```

- seq 명령어 사용:

```
for i in $(seq 1 5); do
    echo "숫자: $i"
done
```

실습

1. "apple", "banana", "orange"를 순회하며 출력해보세요.
2. 1부터 10까지 숫자를 출력하는 for 루프를 작성해보세요.

5.6.2. while 루프

while 루프란?

- 조건이 참인 동안 반복 실행.
- 기본 형식:

```
while [ 조건 ]; do
    # 실행할 코드
done
```

기본 사용법

- 예제:

```
count=1
while [ $count -le 5 ]; do
    echo "카운트: $count"
    count=$((count + 1))
done
```

- 출력:

```
카운트: 1
카운트: 2
카운트: 3
카운트: 4
카운트: 5
```

입력과 결합

- 사용자 입력 받기:

```
while read -p "단어를 입력하세요 (exit로 종료): " word; do
    if [ "$word" = "exit" ]; then
        break
    fi
done
```

```
echo "입력: $word"
done
```

실습

1. 0부터 4까지 출력하는 while 루프를 작성해보세요.
2. 사용자가 "quit"을 입력할 때까지 입력을 받아 출력하는 코드를 작성해보세요.

5.6.3. until 루프

until 루프란?

- 조건이 거짓일 때까지 반복 (조건이 참이면 종료).
- 기본 형식:

```
until [ 조건 ]; do
    # 실행할 코드
done
```

기본 사용법

- 예제:

```
num=0
until [ $num -ge 5 ]; do
    echo "숫자: $num"
    num=$((num + 1))
done
```

- 출력:

```
숫자: 0
숫자: 1
숫자: 2
숫자: 3
숫자: 4
```

while과의 차이

- while: 조건이 참일 때 실행.
- until: 조건이 거짓일 때 실행.

실습

1. 10부터 6까지 감소하며 출력하는 until 루프를 작성해보세요.
2. 변수가 100이 될 때까지 10씩 증가시키며 출력해보세요.

5.6.4. 루프 제어 (break, continue)

break

- 루프를 즉시 종료.
- 예제:

```
for i in {1..10}; do
    if [ $i -eq 5 ]; then
```

```
break
fi
echo "숫자: $i"
done
```

- 출력:

```
숫자: 1
숫자: 2
숫자: 3
숫자: 4
```

continue

- 현재 반복을 건너뛰고 다음 반복으로 이동.
- 예제:

```
for i in {1..5}; do
  if [ $i -eq 3 ]; then
    continue
  fi
  echo "숫자: $i"
done
```

- 출력:

```
숫자: 1
숫자: 2
숫자: 4
숫자: 5
```

실습

1. 1부터 10까지 출력하되, 7에서 break로 종료해보세요.
2. 1부터 5까지 출력하되, 2와 4는 continue로 건너뛰어보세요.

종합 실습

1. for 루프로 1부터 10까지의 합을 계산해 출력하세요.
2. while 루프로 사용자가 "yes"를 입력할 때까지 "계속?"를 묻는 코드를 작성하세요.
3. until 루프로 변수가 0에서 -5가 될 때까지 감소시키며 출력하세요.
4. 1부터 10까지 출력하되, 짝수는 건너뛰고 8에서 종료하는 코드를 작성하세요.

추가 팁

- 산술 연산은 (())로 간소화 가능:

```
i=$((i + 1)) # 또는 ((i++))
```

- 무한 루프:

```
while true; do
  echo "무한 반복"
```



```
sleep 1
done
```

- 파일 읽기와 결합:

```
while read line; do
    echo "라인: $line"
done < file.txt
```

5.7 함수

5.7.1. 함수 정의와 호출

함수란?

- 재사용 가능한 코드 블록.
- Bash에서 함수는 별도의 반환 타입이나 선언 키워드가 필요 없음.

함수 정의

- 두 가지 형식:

```
# 형식 1
function 함수이름 {
    # 코드
}

# 형식 2
함수이름() {
    # 코드
}
```

- 예제:

```
greet() {
    echo "안녕하세요!"
}
```

함수 호출

- 함수 이름만 입력:

```
greet
# 출력: 안녕하세요!
```

주의사항

- 함수는 호출 전에 정의되어야 함.

실습

1. say_hello라는 함수를 정의해 "Hello, World!"를 출력해보세요.
2. 정의한 함수를 호출해보세요.

5.7.2. 매개변수 전달

매개변수란?

- 함수에 전달되는 값 (인자).
- \$1, \$2, ...로 참조.

사용법

- 예제:

```
greet() {  
    echo "안녕, $1님!"  
}  
greet "홍길동"  
# 출력: 안녕, 홍길동님!
```

- 여러 매개변수:

```
add() {  
    echo "$1 + $2 = $(( $1 + $2 ))"  
}  
add 3 5  
# 출력: 3 + 5 = 8
```

특수 변수

- \$#: 매개변수 개수.
- \$@ 또는 \$*: 모든 매개변수.
- 예제:

```
print_all() {  
    echo "매개변수 수: $#"  
    echo "모든 매개변수: $@"  
}  
print_all "a" "b" "c"  
# 출력:# 매개변수 수: 3# 모든 매개변수: a b c
```

실습

1. 이름과 나이를 매개변수로 받아 "이름: [name], 나이: [age]"를 출력하는 함수를 작성해보세요.
2. 세 숫자를 받아 합을 출력하는 함수를 작성해보세요.

5.7.3. 반환값 처리

반환값이란?

- Bash에서 함수의 반환값은 주로 return이나 출력으로 처리.
- return은 상태 코드(0~255)만 반환 가능.

return 사용

- 예제:

```
check() {  
    if [ "$1" -gt 0 ]; then  
        return 0 # 성공
```

```

else
    return 1 # 실패
fi
}
check 5
echo "리턴값: $" # $?는 직전 명령의 종료 상태# 출력: 리턴값: 0

```

출력으로 반환값 전달

- 일반적인 방법:

```

get_sum() {
    echo $(( $1 + $2 ))
}
result=$(get_sum 3 4)
echo "합: $result"
# 출력: 합: 7

```

실습

- 두 수를 비교해 큰 수를 출력으로 반환하는 함수를 작성해보세요.
- 숫자가 짝수면 0, 홀수면 1을 return하는 함수를 작성하고 \$?로 확인해보세요.

5.7.4. 함수 스코프와 전역 변수

스코프란?

- 변수의 유효 범위.
- Bash에서 기본적으로 모든 변수는 **전역 변수**.

전역 변수

- 함수 내에서 선언된 변수도 밖에서 접근 가능:

```

set_var() {
    global_var="함수에서 설정"
}
set_var
echo $global_var
# 출력: 함수에서 설정

```

지역 변수

- local 키워드로 함수 내에서만 유효:

```

test_scope() {
    local local_var="로컬"
    global_var="글로벌"
    echo "함수 안: $local_var, $global_var"
}
test_scope
echo "함수 밖: $local_var, $global_var"
# 출력:# 함수 안: 로컬, 글로벌# 함수 밖: , 글로벌

```

주의사항

- local 없으면 실수로 전역 변수를 덮어쓸 수 있음.

실습

1. 함수 내에서 전역 변수와 지역 변수를 각각 설정하고, 함수 밖에서 값을 확인해보세요.
2. 매개변수를 받아 지역 변수로 저장한 뒤 출력하는 함수를 작성해보세요.

종합 실습

1. 이름과 메시지를 매개변수로 받아 "이름: [name], 메시지: [msg]"를 출력하는 함수를 작성하세요.
2. 두 숫자를 받아 곱셈 결과를 출력으로 반환하는 함수를 작성하고, 결과를 변수에 저장해 출력하세요.
3. 숫자를 받아 10보다 크면 1, 작거나 같으면 0을 return하는 함수를 작성하고 \$?로 확인하세요.
4. 함수 내에서 지역 변수와 전역 변수를 사용해 값을 설정하고, 함수 호출 후 각각의 값을 확인해보세요.

추가 팁

- 함수 이름은 변수와 동일한 규칙 (영문자, 숫자, _, 숫자로 시작 불가).
- 재귀 함수 가능:

```
factorial() {  
    if [ $1 -le 1 ]; then  
        echo 1  
    else  
        echo $(( $1 * $(factorial $(( $1 - 1 ))) ) )  
    fi  
}  
factorial 5 # 출력: 120
```

- 디버깅 시 set -x로 함수 실행 추적 가능.

5.8 파일 및 디렉토리 작업

5.8.1. 파일 읽기와 쓰기 (cat, >, >>)

파일 읽기: cat

- 파일 내용을 출력.
- 기본 사용:

```
cat file.txt
```

- 여러 파일 출력:

```
cat file1.txt file2.txt
```

파일 쓰기: > (덮어쓰기)

- 파일에 내용을 새로 작성 (기존 내용 삭제):

```
echo "새로운 내용" > output.txt
```

- 확인:

```
cat output.txt
# 출력: 새로운 내용
```

파일 추가: >> (이어쓰기)

- 기존 내용에 추가:

```
echo "추가 내용" >> output.txt
cat output.txt
# 출력:# 새로운 내용# 추가 내용
```

실습

1. "Hello, World!"를 test.txt에 작성해보세요.
2. test.txt에 "안녕하세요"를 추가해보세요.
3. cat으로 test.txt 내용을 확인해보세요.

5.8.2. 파일 존재 여부 확인

테스트 연산자

- 파일 상태를 확인하는 조건:
 - e: 파일 또는 디렉토리가 존재.
 - f: 일반 파일인지.
 - d: 디렉토리인지.
 - r, -w, -x: 읽기, 쓰기, 실행 권한 여부.

사용법

- 예제:

```
if [ -f "output.txt" ]; then
    echo "output.txt는 파일입니다."
else
    echo "output.txt가 존재하지 않거나 파일이 아닙니다."
fi
```

- 디렉토리 체크:

```
if [ -d "my_folder" ]; then
    echo "my_folder는 디렉토리입니다."
fi
```

실습

1. "test.txt"가 존재하는지 확인하는 스크립트를 작성해보세요.
2. 현재 디렉토리에 "backup" 폴더가 있는지 확인해보세요.

3. 디렉토리 생성 및 삭제

디렉토리 생성: mkdir

- 기본 사용:

```
mkdir my_folder
```

- 중첩 디렉토리 생성 (-p: 부모 디렉토리까지 생성):

```
mkdir -p parent/child/grandchild
```

디렉토리 삭제: rmdir와 rm

- 빈 디렉토리 삭제 (rmdir):

```
rmdir my_folder
```

- 내용이 있는 디렉토리 삭제 (rm -r):

```
rm -r parent
```

- 확인 요청 추가 (-i):

```
rm -ri parent
```

실습

1. "temp" 디렉토리를 생성해보세요.
2. "temp/subfolder"를 -p 옵션으로 생성해보세요.
3. "temp"를 삭제해보세요.

5.8.4. find와 grep 활용

find 명령어

- 파일 및 디렉토리를 검색.
- 기본 사용:

```
find . -name "file.txt" # 현재 디렉토리에서 file.txt 검색
```

- 파일 유형 지정:

```
find . -type f # 파일만  
find . -type d # 디렉토리만
```

- 이름 패턴:

```
find . -name "*.txt" # .txt로 끝나는 파일
```

grep 명령어

- 파일 내에서 텍스트 검색.
- 기본 사용:

```
grep "hello" file.txt # file.txt에서 "hello" 검색
```

- 대소문자 무시 (-i):

```
grep -i "hello" file.txt
```

- 라인 번호 표시 (-n):

```
grep -n "hello" file.txt
```

find와 grep 결합

- 예제:

```
find . -name "*.txt" -exec grep "hello" {} \; # .txt 파일에서 "hello" 검색
```

실습

1. 현재 디렉토리에서 ".txt"로 끝나는 파일을 찾아보세요.
2. "test.txt"에서 "안녕"이라는 단어를 검색해보세요.
3. .에서 모든 텍스트 파일을 찾아 "Hello"를 검색하는 명령어를 작성해보세요.

종합 실습

1. "data.txt"에 "Line 1", "Line 2", "Line 3"을 작성하세요.
2. "data.txt"가 존재하는지 확인하고, 존재하면 내용을 출력하는 스크립트를 작성하세요.
3. "backup/test" 디렉토리를 생성하고, "data.txt"를 그 안으로 복사(cp)해보세요.
4. "backup" 디렉토리에서 "Line"이라는 단어가 포함된 파일을 찾아보세요.

추가 팁

- 파일 내용 줄 단위 읽기:

```
while read line; do
    echo "라인: $line"
done < file.txt
```

- find 결과 실행:

```
find . -name "*.txt" -exec rm {} \; # .txt 파일 삭제
```

- grep으로 여러 파일 검색:

```
grep "pattern" *.txt
```

9.9 에러 처리와 디버깅

9.9.1. exit 코드와 에러 체크

exit 코드란?

- 스크립트나 명령어의 종료 상태를 나타냄.
- 0: 성공, 1 이상: 실패 (특정 오류에 따라 다름).
- 확인: \$? 변수 사용.

기본 사용법

- 예제:

```
ls /존재하지않는파일
echo "종료 코드: $?"
# 출력: 종료 코드: 2 (파일 없음)
```

- exit로 종료 코드 지정:

```
exit 1 # 비정상 종료
```

에러 체크

- 명령어 성공 여부 확인:

```
if ls myfile.txt > /dev/null 2>&1; then
    echo "파일이 존재합니다."
else
    echo "파일이 없습니다."
fi
```

- \$? 활용:

```
ls myfile.txt
if [ $? -ne 0 ]; then
    echo "오류 발생!"
fi
```

실습

1. 존재하지 않는 파일을 cat으로 열고 \$?를 확인해보세요.
2. 파일이 존재하면 "성공", 없으면 "실패"를 출력하는 스크립트를 작성해보세요.

9.9.2. set 명령어 (-x, -e 등)

set란?

- 셸 동작을 제어하는 옵션 설정.

주요 옵션

- x: 디버깅 모드 (실행된 명령어 출력).

```
set -x
echo "테스트"
ls dir
# 출력:# + echo 테스트# 테스트# + ls dir
```

- e: 오류 발생 시 즉시 종료.

```
set -e
ls /존재하지않는파일 # 여기서 스크립트 종료
echo "이 줄은 실행되지 않음"
```

- u: 정의되지 않은 변수 사용 시 오류.

```
set -u
echo $undefined_var # 오류 발생
```


- 옵션 해제: set +옵션 (예: set +x).

실습

1. set -x를 사용해 간단한 스크립트의 실행 과정을 추적해보세요.
2. set -e를 적용하고, 실패하는 명령어 뒤에 메시지가 출력되지 않음을 확인해보세요.

3. 디버깅 팁과 흔한 실수 피하기

디버깅 팁

- 변수 값 확인: echo로 중간 값 출력.

```
num=5
echo "num 값: $num"
```

- 명령어 분리: 복잡한 파이프라인을 단계별로 테스트.

```
find . -name "*.txt" | grep "test"
# 분리:
find . -name "*.txt"
echo "위 결과를 grep으로 필터링"
```

- 로그 파일: 출력 리다이렉션으로 기록.

```
echo "오류 발생" >> debug.log
```

- bash -x: 스크립트 실행 시 디버깅 모드.

```
bash -x myscript.sh
```

흔한 실수와 해결법

1. 공백 문제:
 - 잘못된 예: if [\$var=5]
 - 수정: if ["\$var" = 5] (공백과 따옴표 필수).
2. 변수 미초기화:
 - 오류: echo \$undefined
 - 해결: echo \${undefined:-기본값}.
3. 따옴표 누락:
 - 잘못된 예: if [\$var = "test string"]
 - 수정: if ["\$var" = "test string"].
4. 잘못된 경로:
 - 확인: pwd, ls -l로 경로 점검.

실습

1. 변수에 공백이 포함된 문자열을 if로 비교해보고, 따옴표 없이 오류가 나는 경우를 확인해보세요.
2. set -x로 3줄짜리 스크립트를 실행해 디버깅 출력을 확인해보세요.
3. 존재하지 않는 변수 사용 시 set -u로 오류를 발생시켜보세요.

종합 실습

1. 파일이 존재하는지 확인하고, 없으면 "파일 없음"을 출력 후 exit 1로 종료하는 스크립트를 작성하세요.
2. set -e를 사용해 명령어가 실패하면 중단되는 스크립트를 작성해보세요.
3. 아래 스크립트에서 오류를 찾아 디버깅해보세요:

```
num=10
if [ $num > 5 ]; then
    echo "5보다 큼"
fi
```

- 힌트: 비교 연산자 문제.

추가 팁

- **트랩 사용:** 종료 시 정리 작업.

```
trap 'echo "스크립트 종료"; exit' EXIT
```

- **의미 있는 종료 코드:** 사용자 정의 오류 코드 사용.

```
exit 10 # 파일 없음
exit 20 # 권한 오류
```

- **셸 체크 도구:** shellcheck로 스크립트 점검.

```
shellcheck myscript.sh
```

9.10 종합 실습

9.10.1. 파일 백업 스크립트 (Backup Script)

```
#!/bin/bash

# 스크립트의 목적: 지정된 디렉토리를 타임스탬프와 함께 백업하는 스크립트# 백업할 소스 디렉토리 경로를 변수에 저장
SOURCE_DIR="/home/user/documents"

# 백업 파일이 저장될 디렉토리 경로
BACKUP_DIR="/home/user/backups"

# 현재 날짜와 시간을 타임스탬프 형식으로 저장 (예: 202504061014)
TIMESTAMP=$(date +%Y%m%d%H%M)

# 백업 파일 이름: backup_타임스탬프.tar.gz 형식
BACKUP_FILE="backup_${TIMESTAMP}.tar.gz"

# 소스 디렉토리가 존재하는지 확인
if [ ! -d "$SOURCE_DIR" ]; then
    # 디렉토리가 없으면 에러 메시지 출력 후 종료
    echo "Error: Source directory $SOURCE_DIR does not exist."
    exit 1
fi
```

```

# 백업 디렉토리가 존재하지 않으면 생성
if [ ! -d "$BACKUP_DIR" ]; then
    mkdir -p "$BACKUP_DIR"
    echo "Created backup directory: $BACKUP_DIR"
fi

# tar 명령어로 소스 디렉토리를 압축하여 백업 디렉토리에 저장
tar -czvf "$BACKUP_DIR/$BACKUP_FILE" "$SOURCE_DIR"

# 백업 성공 여부 확인
if [ $? -eq 0 ]; then
    # $?는 마지막 명령어의 종료 코드를 반환 (0은 성공)
    echo "Backup completed successfully: $BACKUP_DIR/$BACKUP_FILE"
else
    echo "Backup failed!"
    exit 1
fi

```

설명: 이 스크립트는 파일 백업을 자동화하는 데 자주 사용됩니다. 타임스탬프를 추가해 백업 파일을 고유하게 만들고, 디렉토리 존재 여부를 체크하며 에러 처리를 포함합니다.

2. 시스템 모니터링 스크립트 (System Monitoring Script)

```

#!/bin/bash

# 스크립트의 목적: CPU, 메모리, 디스크 사용량을 모니터링하고 출력# CPU 사용량을 퍼센트로 계산 (top 명령어 사용)
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 + $4}')

# 메모리 사용량을 퍼센트로 계산 (free 명령어 사용)
MEM_TOTAL=$(free -m | awk '/Mem:/ {print $2}')
MEM_USED=$(free -m | awk '/Mem:/ {print $3}')
MEM_USAGE=$(echo "scale=2; $MEM_USED/$MEM_TOTAL*100" | bc)

# 디스크 사용량을 퍼센트로 계산 (df 명령어 사용)
DISK_USAGE=$(df -h | awk 'NR==2 {print $5}' | sed 's/%//')

# 결과 출력
echo "System Monitoring Report - $(date)"
echo "-----"
echo "CPU Usage: $CPU_USAGE%"
echo "Memory Usage: $MEM_USAGE%"
echo "Disk Usage: $DISK_USAGE%"

# 디스크 사용량이 90% 이상이면 경고 메시지 출력
if [ "$DISK_USAGE" -ge 90 ]; then
    echo "WARNING: Disk usage is critically high!"
fi

```

설명: 시스템 상태를 모니터링하는 스크립트로, CPU, 메모리, 디스크 사용량을 출력합니다. 조건문을 사용해 임계값 초과 시 경고를 띄웁니다. 실시간 모니터링에 유용합니다.

3. 사용자 입력 계산기 (Simple Calculator Script)

```

#!/bin/bash

```

```

# 스크립트의 목적: 사용자 입력을 받아 기본 계산을 수행# 사용자에게 첫 번째 숫자 입력 요청
echo -n "Enter the first number: "
read NUM1

# 사용자에게 두 번째 숫자 입력 요청
echo -n "Enter the second number: "
read NUM2

# 연산 선택 메뉴 출력
echo "Choose an operation:"
echo "1) Addition (+)"
echo "2) Subtraction (-)"
echo "3) Multiplication (*)"
echo "4) Division (/)"
echo -n "Enter your choice (1-4): "
read CHOICE

# case 문으로 선택된 연산 수행
case $CHOICE in
    1)
        RESULT=$((NUM1 + NUM2))
        echo "$NUM1 + $NUM2 = $RESULT"
        ;;
    2)
        RESULT=$((NUM1 - NUM2))
        echo "$NUM1 - $NUM2 = $RESULT"
        ;;
    3)
        RESULT=$((NUM1 * NUM2))
        echo "$NUM1 * $NUM2 = $RESULT"
        ;;
    4)
        # 나누기에서 0으로 나누는 경우 체크
        if [ "$NUM2" -eq 0 ]; then
            echo "Error: Division by zero is not allowed!"
            exit 1
        fi
        RESULT=$(echo "scale=2; $NUM1 / $NUM2" | bc)
        echo "$NUM1 / $NUM2 = $RESULT"
        ;;
    *)
        echo "Invalid choice! Please select 1-4."
        exit 1
        ;;
esac

```

설명: 사용자와 상호작용하며 간단한 계산을 수행합니다. case 문과 입력 처리(read)를 활용하며, 나누기에서 0으로 나누는 오류를 방지합니다.

4. 파일 정리 스크립트 (File Cleanup Script)

```

#!/bin/bash

# 스크립트의 목적: 지정된 디렉토리에서 오래된 파일을 삭제# 정리할 디렉토리 경로
TARGET_DIR="/home/user/downloads"

```

```
# 삭제할 파일의 기준 일수 (예: 30일 이상 된 파일)
DAYS=30

# 디렉토리가 존재하는지 확인
if [ ! -d "$TARGET_DIR" ]; then
    echo "Error: Directory $TARGET_DIR does not exist."
    exit 1
fi

# find 명령어로 30일 이상 된 modernity 된 파일 삭제
echo "Cleaning up files older than $DAYS days in $TARGET_DIR..."
find "$TARGET_DIR" -type f -mtime +$DAYS -exec rm -v {} \;

# 삭제된 파일 수 확인
DELETED_COUNT=$(find "$TARGET_DIR" -type f -mtime +$DAYS | wc -l)
echo "$DELETED_COUNT files deleted."
```

설명: 오래된 파일을 정리하는 스크립트로, find 명령어를 사용해 지정된 일수 이상 된 파일을 삭제합니다. 시스템 관리에서 자주 활용됩니다.

5. 로그 파일 분석 스크립트 (Log File Analysis Script)

```
#!/bin/bash

# 스크립트의 목적: 로그 파일에서 오류 메시지를 찾아 요약# 분석할 로그 파일 경로
LOG_FILE="/var/log/syslog"

# 출력할 결과 파일
OUTPUT_FILE="error_summary.txt"

# 로그 파일이 존재하는지 확인
if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Log file $LOG_FILE does not exist."
    exit 1
fi

# "error" 또는 "fail"이 포함된 줄을 찾아 출력 파일에 저장
echo "Analyzing log file: $LOG_FILE" > "$OUTPUT_FILE"
echo "Errors found on $(date):" >> "$OUTPUT_FILE"
echo "-----" >> "$OUTPUT_FILE"
grep -i "error\|fail" "$LOG_FILE" >> "$OUTPUT_FILE"

# 오류 수 계산
ERROR_COUNT=$(grep -i "error\|fail" "$LOG_FILE" | wc -l)

# 결과 요약 출력
echo "Analysis complete. Found $ERROR_COUNT errors/failures."
echo "Details saved to $OUTPUT_FILE"
```

9.11 크론잡으로 스크립트 자동화

9.11.1. 크론잡이란?

- **정의:** 크론(Cron)은 유닉스 계열 시스템에서 주기적으로 작업을 실행하기 위한 스케줄링 도구입니다.
- **용도:** 백업, 시스템 모니터링, 로그 정리 등 반복 작업 자동화.
- **작동 방식:** crontab 파일에 작업 스케줄을 정의하고, 크론 데몬이 이를 실행.

2. 실제 예제: 자동 백업 스크립트와 크론잡 설정

아래는 매일 밤 2시에 디렉토리를 백업하는 스크립트와 크론잡 설정 예제입니다.

백업 스크립트 (backup.sh)

```
#!/bin/bash

# 스크립트 목적: 지정된 디렉토리를 타임스탬프와 함께 백업하고 로그를 기록# 백업할 소스 디렉토리 경로
SOURCE_DIR="/home/user/documents"

# 백업 파일이 저장될 디렉토리 경로
BACKUP_DIR="/home/user/backups"

# 로그 파일 경로 (크론 실행 결과를 기록)
LOG_FILE="/home/user/backups/backup_log.txt"

# 현재 날짜와 시간을 타임스탬프 형식으로 저장 (예: 20250406_0200)
TIMESTAMP=$(date +%Y%m%d_%H%M)

# 백업 파일 이름: backup_타임스탬프.tar.gz 형식
BACKUP_FILE="backup_${TIMESTAMP}.tar.gz"

# 로그에 작업 시작 시간 기록
echo "[$(date)] Backup started for $SOURCE_DIR" >> "$LOG_FILE"

# 소스 디렉토리가 존재하는지 확인
if [ ! -d "$SOURCE_DIR" ]; then
    # 디렉토리가 없으면 에러 메시지를 로그에 기록하고 종료
    echo "[$(date)] Error: Source directory $SOURCE_DIR does not exist." >> "$LOG_FILE"
    exit 1
fi

# 백업 디렉토리가 존재하지 않으면 생성
if [ ! -d "$BACKUP_DIR" ]; then
    mkdir -p "$BACKUP_DIR"
    echo "[$(date)] Created backup directory: $BACKUP_DIR" >> "$LOG_FILE"
fi

# tar 명령어로 소스 디렉토리를 압축하여 백업 디렉토리에 저장# 출력과 에러를 로그 파일에 리다이렉트
tar -czvf "$BACKUP_DIR/$BACKUP_FILE" "$SOURCE_DIR" >> "$LOG_FILE" 2>&1

# 백업 성공 여부 확인
if [ $? -eq 0 ]; then
    # $?는 마지막 명령어의 종료 코드를 반환 (0은 성공)
    echo "[$(date)] Backup completed: $BACKUP_DIR/$BACKUP_FILE" >> "$LOG_FILE"
else
    echo "[$(date)] Backup failed!" >> "$LOG_FILE"
    exit 1
fi
```

크론잡 설정

1. 스크립트 실행 권한 부여:

```
chmod +x /home/user/scripts/backup.sh
```

2. 크론잡 편집:

```
crontab -e
```

3. 아래 줄을 추가 (매일 새벽 2시 실행):

```
0 2 * * * /home/user/scripts/backup.sh
```

4. 크론잡 저장 후 확인:

```
crontab -l
```

3. 크론 사용법 상세 설명

크론탭(Crontab) 기본 구조

- **형식:** 분 시 일 월 요일 명령어
- **필드 설명:**
 - 분: 0-59
 - 시: 0-23
 - 일: 1-31
 - 월: 1-12
 - 요일: 0-7 (0과 7은 일요일)
 - 명령어: 실행할 스크립트 또는 명령
- **특수 문자:**
 - : 모든 값 (예: 모든 시간)
 - ,: 값 나열 (예: 1,3,5)
 - : 범위 (예: 1-5)
 - /: 간격 (예: */5는 5분마다)

크론탭 예제

크론탭 항목	의미
0 0 * * *	매일 자정에 실행
*/15 * * * *	15분마다 실행
0 9-17 * * 1-5	평일 오전 9시부터 오후 5시까지 매시 실행
0 2 1 * *	매월 1일 새벽 2시에 실행
30 23 * * 0	매주 일요일 23:30에 실행

크론 명령어

- `crontab -e`: 크론탭 파일 편집
- `crontab -l`: 현재 크론탭 목록 확인
- `crontab -r`: 크론탭 삭제
- `sudo crontab -e`: 루트 권한으로 크론탭 편집 (시스템 작업용)

크론 로그 확인

- 크론 실행 로그는 일반적으로 /var/log/syslog 또는 /var/log/cron에 기록됨.
- 확인 명령어:

```
grep CRON /var/log/syslog
```

크론 사용 시 주의점

1. **절대 경로 사용:** 크론은 상대 경로를 인식하지 않으므로 스크립트와 파일 경로를 절대 경로로 지정.
2. **환경 변수:** 크론은 기본 환경 변수가 제한적이므로, 필요한 경우 스크립트 내에서 정의 (예: PATH 설정).

```
# 스크립트 상단에 추가
PATH=/usr/local/bin:/usr/bin:/bin
```

3. **출력 리다이렉션:** 크론은 기본적으로 결과를 이메일로 보내므로, 로그 파일로 리다이렉션 권장.

```
0 2 * * * /home/user/scripts/backup.sh >> /home/user/backups/cron_log.txt 2>&1
```

4. 실습 과제

1. **기본 실습:** 위 백업 스크립트를 로컬 환경에 저장하고, 매 5분마다 실행되도록 크론잡 설정.

```
/5 * * * * /path/to/backup.sh
```

2. **응용 과제:** 스크립트를 수정해 7일 이상 된 백업 파일을 자동 삭제하도록 추가.

```
# 스크립트 하단에 추가
find "$BACKUP_DIR" -type f -mtime +7 -exec rm -v {} \; >> "$LOG_FILE" 2>&1
echo "[$(date)] Old backups (older than 7 days) cleaned up." >> "$LOG_FILE"
```

5. 강의 마무리

- **요약:** 크론잡은 Bash 스크립트와 결합해 강력한 자동화 도구로 사용됩니다. 스케줄링 문법과 로그 관리가 핵심입니다.
- **질문:** "어떤 작업을 자동화하고 싶나요?"로 토론 유도.
- **추가 학습:** man crontab, 온라인 크론 생성기 (예: crontab.guru) 탐색 권장.

추가 팁

- **강의 중 시연:** 간단한 스크립트를 작성하고 크론잡을 설정해 실시간으로 실행 결과를 보여주세요.
- **문제 해결 연습:** "스크립트가 실행되지 않을 때 확인할 점" (권한, 경로, 로그 등)을 다뤄보세요.